FocusPet 零基础开发教程

从零开始，用 Swift 构建一只陪你专注的桌面宠物

15 篇正文 · 约 4000 行真实源码 · 每行代码都可对照

2026 年 9 月

目录

00 · 开篇：你即将亲手构建什么 3
01 · Swift 语法零基础速成 5
02 · macOS 应用解剖：从启动到窗口 10
03 · 架构总览：11 个类的分工地图 14
04 · 透明悬浮窗：宠物的舞台 17
05 · 动画引擎：序列帧与省电重绘 20
06 · 行为调度：让宠物「活」起来 24
07 · 交互系统：点击、拖拽、双击 27
08 · 番茄钟引擎：状态机思维 33
09 · 状态栏图标、全局快捷键与系统通知 39
10 · 设置面板与数据持久化 45
11 · 统计系统：JSON 存储与快照算法 51
12 · AI 素材生产管线：从一句话到 144 帧动画 58
13 · 构建、打包与发布：从 swift run 到 /Applications 63
14 · 加餐：投喂饥饿系统与省电优化 69
FocusPet 零基础开发教程 · 总索引 76

00 · 开篇：你即将亲手构建什么

FocusPet 零基础开发教程 · 第 0 篇

这个项目是什么

一只住在你的 Mac 桌面上的小猫（或小狗）。它会：

- 在屏幕上**慢悠悠散步**（猫）或**蹦跳撒欢**（狗），偶尔全屏乱窜
- 你**按住拖动**它，它会挥手挣扎；松手落地弹一下
- 你**单击**它，它吃鱼/啃骨头；**双击**它，它开心地庆祝、飘爱心
- 你**开始专注**（番茄钟 25 分钟），它头顶出现倒计时时钟，安静看书
- 专注结束，它**跑到屏幕正中央**为你庆祝，然后提醒你休息
- 超过 2 小时没喂它，它会委屈地讨食；专注时它不打扰，只发一条通知
- 状态栏有它的图标，可以切猫切狗、看统计、改设置

这不是玩具demo，而是一个**完整发布的 macOS 应用**：有窗口系统、动画引擎、状态机、数据持久化、系统通知、开机自启、Release 构建。

教程怎么读

篇	内容	需要什么基础
01	Swift 语法零基础速成	会任意一门语言更好，不会也能跟
02	macOS 应用解剖：程序怎么跑起来	读完 01
03	架构总览：11 个类怎么分工	读完 02
04~09	每个子系统一篇（窗口/动画/行为/交互/番茄钟/状态栏）	读完 03
10~11	设置面板 & 统计系统（SwiftUI 入门）	读完 04
12	AI 素材生产管线（提示词→图集→切帧）	独立
13	构建、打包、发布、排错	读完 03
14	加餐：投喂/饥饿系统 + 省电优化思路	读完 05、06

核心思路（先建立直觉）

整个应用其实只回答三个问题：

1. **画什么**——一个透明窗口里，按时间顺序切换 8 张 PNG 图片（帧动画）
2. **什么时候画**——用「状态机 + 定时器」决定此刻该播哪套动画、该不该移动

3. **怎么记住**——UserDefaults 存设置，JSON 文件存专注记录

所有「智能」的行为（散步、讨食、庆祝）都是**随机数 + 计时器**的组合，没有 AI、没有机器学习。理解了这一点，你就不会被代码量吓到。

开发环境

- 一台 Mac（macOS 14+）
- Xcode 命令行工具：`xcode-select --install`（不用装完整的 Xcode）
- 验证：`swift --version` 能输出版本号即可

建议的学习方式

1. 先通读 01~03，建立整体地图
2. 然后**对照源码读**每一篇（教程里贴了关键代码和文件路径）
3. 每读完一篇，自己改一个小地方（改时长、改概率、改文案），跑起来看效果
4. 全部读完后，尝试「给宠物加第 3 只」——那是最好的毕业考试

源码位置：`FocusPet/Sources/FocusPet/` 下每个 `.swift` 文件对应一个类，文件名 = 类名，非常好找。

01 · Swift 语法零基础速成

FocusPet 零基础开发教程 · 第 1 篇

本篇只讲读懂本项目必需的 Swift 语法，30 分钟建立感觉。Swift 由 Apple 设计，目标是「安全 + 表达力强」，很多语法和 JavaScript/Python 有相似之处。

1. 常量和变量

```
let name = "咪咪" // let = 常量，赋值后不能改（默认优先用 let）
var age = 3 // var = 变量，可以改
age = 4

// 类型标注（一般不写，编译器自动推断）
let explicit: String = "旺财"
let count: Int = 8
let ratio: Double = 0.75
let visible: Bool = true
```

重要习惯：Swift 里 `let` 优先。本项目 90% 的声明都是 `let`。

2. 函数

```
// 无返回值
func greet() {
print("你好")
}

// 有参数、有返回值
func add(_ a: Int, to b: Int) -> Int {
return a + b
}
add(3, to: 5) // 8。参数名前加 _ 表示调用时不写参数名

// 参数标签（Swift 特色，提高可读性）
func play(_ action: PetAction, completion: (() -> Void)? = nil) { ... }
play(.idle) // completion 有默认值，可省略
play(.wave) { print("播完了") } // 尾随闭包语法
```

3. 可选类型（Optional）最重要的概念

Swift 用 `?` 表示「可能没有值」，用 `!` 或 `if let` 拆包：

```
var petName: String? = "咪咪" // 可能是 String，可能是 nil
petName = nil // 表示"没有名字"

// 安全拆包方式一：if let（推荐）
if let name = petName {
print(name) // 只有非 nil 才进这里，name 是普通 String
} else {
print("没名字")
}

// 方式二：guard let（提前退出，函数里常用）
func feed(_ pet: PetController?) {
guard let pet else { return } // pet 为 nil 直接返回
pet.feed() // 这里 pet 一定是非 nil
}

// 方式三：可选链 ?.（调用链上任何一环为 nil 就整个返回 nil，不崩溃）
window?.performDrag(with: event)

// 方式四：强制拆包 !（确定有值时用，本项目刻意几乎不用）
```

为什么重要：macOS API 里到处都是可选（`statusItem.button?`、`window?.frame`），读不懂 `?` 就读不懂源码。

4. 闭包（Closure）

就是「可以当变量传的代码块」，类似 JS 的匿名函数：

```
// 完整写法
let sayHi = { (name: String) -> Void in
print("你好 \(name)")
}
sayHi("咪咪")

// 简写（项目里常见）
DispatchQueue.main.asyncAfter(deadline: .now() + 5) {
self.play(.idle) // 参数是闭包时，可用尾随闭包语法放在括号外
}

// 逃逸闭包 @escaping（闭包在函数返回后才执行，必须标注）
var onComplete: (() -> Void)? // 项目里大量这种"回调槽位"
```

回调模式（本项目核心通信方式）：

```
final class FocusEngine {
var onTick: ((TimeInterval) -> Void)? // 预留一个"插槽"
}
engine.onTick = { remaining in // 使用方把代码插进去
label.text = "\(remaining)"
}
```

类与类之间不互相引用，而是「我定义回调插槽，你往里插代码」——这是整个项目的解耦哲学。

5. 结构体与类

```
struct AnimationSpec { // struct = 值类型，拷贝即复制（本项目的小数据都用它）
let prefix: String
let count: Int
let frameDuration: Double
}

final class PetController { // class = 引用类型，拷贝的是"指针"（管理窗口/定时器等"活物"用它）
let species: PetSpecies // let 属性：初始化后不可换，但对象内部状态可变
private var timer: Timer? // private = 仅类内部可见
}
```

判断标准：**只是数据**用 struct，**有生命周期、要改别人状态**用 class。`final` 表示不许继承，项目全部加了。

6. 枚举与关联（项目的动作系统）

```
enum PetAction: String, CaseIterable {
case idle, wave, sleep, focus, celebrate, sad, run, walk, eat
}
// 用法：play(.walk)
// CaseIterable 让枚举可以 for 循环遍历

// 带计算属性的枚举（星座篇用到）
enum Zodiac: Int, CaseIterable {
case aries, taurus
var name: String {
switch self {
case .aries: return "白羊座"
case .taurus: return "金牛座"
}
}
}
```

7. 字典与数组

```
let frames: [NSImage] = [...] // 数组
let specs: [PetAction: AnimationSpec] = [ // 字典（键值对）
.idle: AnimationSpec(prefix: "idle", count: 8, ...),
.walk: AnimationSpec(prefix: "walk", count: 8, ...)
]
let walkSpec = specs[.walk] // 返回 Optional！因为键可能不存在

// 项目里用了 force unwrap（!）的地方：species.specs[.idle]!
// 因为我们 100% 确定每个动作都配了素材，所以敢用。新手建议先用 if let。
```

8. 字符串插值与格式化

```
let name = "咪咪"
print("你好 \(name)") // 插值
let text = String(format: "%02d:%02d", 8, 5) // "08:05"（倒计时用它补零）
```

9. 时间

```
let now = Date() // 当前时间
let timestamp = now.timeIntervalSince1970 // 时间戳（存 UserDefaults 用）
let later = Date().addingTimeInterval(3600) // 一小时后
CACurrentMediaTime() // 动画专用的高精度时钟（推荐）
```

10. 单例与静态成员

```
final class StatsStore {
static let shared = StatsStore() // 全局唯一实例
private init() {} // 私有化构造，外部只能 shared.xxx
}
// 使用：StatsStore.shared.add(record)
```

11. guard / switch / for 速览

```
guard !frames.isEmpty else { return } // 不满足条件就滚，满足才往下走

switch Int.random(in: 0..<10) { // 区间 0~9
case 0...4: stroll() // 区间匹配
case 5...6: wave()
default: break
}

for i in 0..<8 { load("frame_\(i)") } // 0~7 循环
```

12. self 与弱引用 [weak self]

```
// 定时器/闭包会"持有" self，若 self 也持有它们 → 循环引用 → 内存泄漏
roamTimer = Timer.scheduledTimer(withTimeInterval: delay) { [weak self] _ in
self?.performRoam() // self 变可选，对象被销毁时自动变 nil，不会崩溃
}
```

最小练习

打开终端 `swift` 进入 REPL，敲：

```
let name = "咪咪"
print("我是\(name)")
var hungry = true
if hungry { print("喂我") }
func feed(_ n: String) -> String { "\(n)吃饱了" }
print(feed(name))
```

能跑通，本篇毕业。下一篇我们看看 macOS 应用到底是怎么启动的。

02 · macOS 应用解剖：从启动到窗口

FocusPet 零基础开发教程 · 第 2 篇

1. 一个 macOS 应用是怎么启动的

你双击 FocusPet.app，系统实际做了这些事：

```
双击 .app
→ 找到 Contents/Info.plist（应用的"身份证"：名字、版本、图标、主类）
→ 启动 Contents/MacOS/FocusPet（真正的可执行文件）
→ 创建 NSApplication 单例（应用的"总开关"）
→ 调用 AppDelegate 的 applicationDidFinishLaunching
→ 我们的代码从这里开始接管
```

对应到代码（AppDelegate.swift）：

```
@main // 程序入口标记（整个项目只有这一个）
final class AppDelegate: NSObject, NSApplicationDelegate {
func applicationDidFinishLaunching(_ notification: Notification) {
applyPetSelection() // 创建宠物
startHungerCheck() // 启动饥饿计时
// ... 组装菜单、热键、通知授权
}
}
```

AppDelegate 是应用的"总导演"：它不画画、不播动画，只负责在正确的时刻调正确的人。

2. 为什么用 SwiftPM 而不是 Xcode 工程

项目根目录的 Package.swift 定义了构建方式：

```
let package = Package(
name: "FocusPet",
platforms: [.macOS(.v14)],
targets: [
.executableTarget(
name: "FocusPet",
path: "Sources/FocusPet",
exclude: ["Info.plist"],
resources: [.copy("PetAssets")] // 把素材文件夹打进程序包
)
]
)
```

好处：swift build 一条命令编译，不需要 .xcodeproj 那些复杂配置；素材通过 .copy("PetAssets") 声明，编译后自动成为 Bundle 资源，代码里用 Bundle.module.url(forResource:...) 就能找到。

3. AppKit vs SwiftUI：本项目为什么混着用

	AppKit（1980s 至今）	SwiftUI（2019+）
本质	命令式：你告诉系统每一步怎么做	声明式：你描述"要什么样"，系统自己算怎么变
透明悬浮窗	完全掌控（NSPanel + 自绘）	做不了这种像素级控制
表单/滑杆/图表	啰嗦	几行搞定

所以本项目分工：

- **宠物本体**（PetView）= 纯 AppKit，每一帧都自己 `draw(_:)`
- **设置/统计窗口**（SettingsView/StatsView）= SwiftUI，用 `NSHostingView` 包一层塞进 AppKit 窗口

```
let host = NSHostingView(rootView: SettingsView(model: settings))
panel.contentView = host // SwiftUI 视图住进 AppKit 窗口
```

4. Info.plist：应用的身份证

`Sources/FocusPet/Info.plist` 里几个关键键：

键	值	作用
`CFBundleIdentifier`	local.focuspet.app	唯一 ID（自启、通知都靠它）
`CFBundleShortVersionString`	0.7	版本号（关于窗口显示）
`CFBundleIconFile`	AppIcon	Dock/访达图标
`LSUIElement`	true	**关键！**应用不在 Dock 显示、没有主菜单栏（状态栏应用标配）

`LSUIElement=true` 是"为什么这个 app 没有 Dock 图标"的答案——它是"辅助应用"（accessory app）。这个设定也带来了一个著名的坑（第 9 篇讲：菜单弹出不响应，因为 accessory 应用默认不激活）。

5. NSPanel vs NSWindow

macOS 有两种窗口：

- `NSWindow`：正经窗口（有标题栏、可最小化）
- `NSPanel`：特殊窗口——可以**无边框、透明、悬浮在所有 app 之上、不抢焦点**

宠物显然要后者。创建代码（`PetController.swift`）：

```
panel = NSPanel(
contentRect: frame,
styleMask: [.borderless, .nonactivatingPanel], // 无边框 + 点它不抢前台
backing: .buffered,
defer: false
)
panel.backgroundColor = .clear // 透明底色（配合 AllowsTransparency 式效果）
panel.isOpaque = false
panel.level = .floating // 悬浮层：永远在普通窗口之上
panel.collectionBehavior = [.canJoinAllSpaces, .fullScreenAuxiliary, .stationary]
// 可以出现在所有桌面/全屏 app 之上，且切 Space 时不动
```

6. 事件循环（RunLoop）：一切动画和交互的地基

macOS 应用启动后进入一个永不退出的循环：

```
while true {
取出一个事件（鼠标点击 / 键盘 / 定时器到点 / 屏幕刷新）
分发给对应对象处理
没有事件就休眠省电
}
```

这就是为什么我们可以**同时**有：CVDisplayLink 动画、N 个 Timer、鼠标拖拽、菜单——它们都是事件循环里的不同事件源。

7. 内存管理：ARC 与循环引用

Swift 自动管理内存（ARC）：对象没有"人"引用它时自动销毁。

```
final class PetController {
private var roamTimer: Timer?
// Timer 会持有闭包，闭包持有 self（若写 [self]）
// self 持有 timer → 互相持有 → 谁也别想销毁（内存泄漏）
}
```

解法就是第 1 篇的 `[weak self]`。项目里所有定时器闭包都用了它。

8. 单实例保护

`applicationWillFinishLaunching` 里有个实用技巧：

```
let others = NSRunningApplication
.runningApplications(withBundleIdentifier: bundleID)
.filter { $0.processIdentifier != getpid() }
if !others.isEmpty { exit(0) } // 已在运行 → 自己退出
```

否则双击两次 app 会出现两只宠物叠在一起。

小结

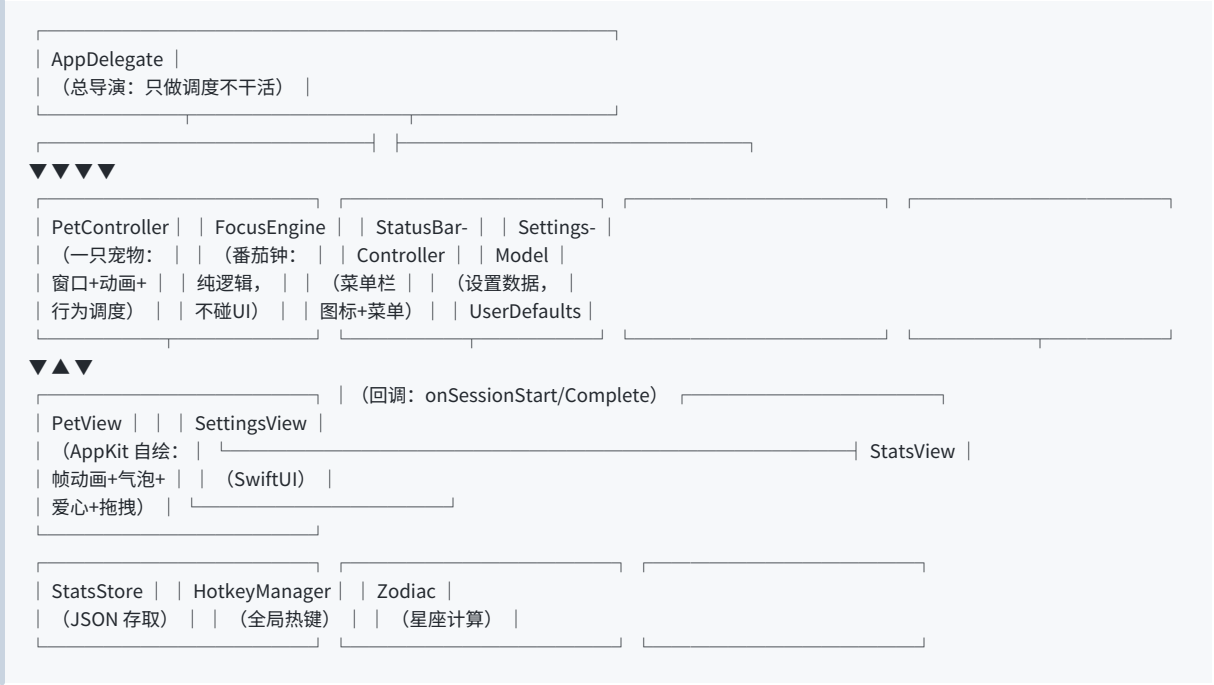
> 双击 .app → Info.plist → NSApplication → AppDelegate
> ↓
> AppKit 搭骨架（窗口/状态栏/菜单）
> PetView(AppKit) 画动画 SwiftUI 画面板
> 类与类用"回调闭包"通信
> ↓
> RunLoop 让一切同时转起来

下一篇：11 个类怎么分工——读源码前的地图。

03 · 架构总览：11 个类的分工地图

FocusPet 零基础开发教程 · 第 3 篇

全景图



逐类速览

类	文件	一句话职责	核心 API/机制
AppDelegate	AppDelegate.swift	总装线：启动一切、转发事件	`applicationDidFinishLaunching`
PetController	PetController.swift	一只宠物的全部：窗口、播动画、乱窜、讨食	Timer、DispatchWorkItem
PetView	PetView.swift	纯绘制：把当前帧画到透明窗口	`draw(_:)`、CVDisplayLink
AnimationSpec	PetView.swift 顶部	一套动画的参数（前缀/帧数/帧时长/循环数）	struct
PetSpecies	PetController.swift 顶部	一个物种：名字、图标、8 套动画的规格表	struct + 静态常量 cat/dog
FocusEngine	FocusEngine.swift	番茄钟状态机	Timer + 4 个回调
StatusBarController	StatusBarController.swift	菜单栏 图标和菜单弹出	NSStatusItem
HotkeyManager	HotkeyManager.swift	Shift+Cmd+P 全局快捷键	Carbon RegisterEventHotKey
SettingsModel	SettingsModel.swift	所有设置项 + UserDefaults 持久化	ObservableObject
StatsStore	StatsStore.swift	专注记录的 JSON 存取 + 统计计算	Codable + JSONEncoder
Zodiac	Zodiac.swift	生日→星座、星座匹配度	enum + switch

设计模式 1：回调解耦（最重要的模式）

类之间**不互相持有**，只留"插槽"：

```
// FocusEngine 不知道 PetController 存在，只广播"会话开始了"
final class FocusEngine {
var onSessionStart: ((Session) -> Void)?
}
// AppDelegate 把两边接上：
engine.onSessionStart = { [weak self] session in
self?.pet?.onSessionStart(isRest: session.kind.isRest)
}
```

好处：番茄钟引擎可以单独测试、单独复用；改宠物不影响计时逻辑。新增功能时，大部分情况是"在中间加一根线"，而不是"改两边的代码"。

设计模式 2：状态由"数据"驱动

宠物的所有外观状态都收在两个枚举里：

```
enum PetAction: String, CaseIterable {
case idle, wave, sleep, focus, celebrate, sad, run, walk, eat
}
enum Phase { case idle, running, paused } // FocusEngine 的
```

菜单标题、动画选择、气泡显示，全部是对这两个枚举的 switch。想加新状态（比如"洗澡"），只需：枚举加一个 case → 素材加一套 → switch 补几个分支。

设计模式 3：单一职责到极致

注意 PetController 和 PetView 的分工：

- **Controller 决定"播什么、去哪"**（调用 play(.walk)、设置目标点）
- **View 只负责"画出来"**（拿到帧数组就画，不知道为什么要画）

这就是经典的 MVC（Model-View-Controller）。View 永远不写业务逻辑，所以换皮（换物种）不用动 View。

数据流：一次专注的完整旅程

```
用户点「开始专注」
→ AppDelegate.startFocus()
→ FocusEngine.startFocus()：phase=running，每秒 onTick
→ AppDelegate：onTick 里更新气泡文字 "24:59"
→ （25分钟后）onSessionComplete
→ AppDelegate：notify() 发系统通知 + NSSound 提示音
→ AppDelegate：pet?.celebrateAtScreenCenter { engine.startRest() }
→ PetController：跑过去 → bounce → play(.celebrate)
→ 庆祝播完 → 回调触发 → engine.startRest() 进入休息
→ StatsStore.add(record) 落盘
```

每一根箭头都是一次回调调用。跟着这个流程读源码，30 分钟就能串起整个项目。

文件导览（读代码顺序建议）

```
1. PetView.swift 的 AnimationSpec ← 最小数据结构，热身
2. FocusEngine.swift ← 141 行，独立好懂，状态机样板
3. StatsStore.swift ← 112 行，JSON 持久化样板
4. PetController.swift ← 最大文件，重点啃行为调度
5. AppDelegate.swift ← 看"线"是怎么接的
6. PetView.swift ← 最后看绘制细节
```

下一篇进入第一个核心子系统：透明悬浮窗。

04 · 透明悬浮窗：宠物的舞台

FocusPet 零基础开发教程 · 第 4 篇
对应源码：PetController.swift 的 init、setScale；PetView.swift 的 petArea、draw

1. 目标效果

一个**没有标题栏、没有背景色、永远浮在所有应用之上、点它不抢焦点**的小窗口，里面住着猫。这在 macOS 上的配方是：

```
panel = NSPanel(
contentRect: frame,
styleMask: [.borderless, .nonactivatingPanel],
backing: .buffered,
defer: false
)
panel.backgroundColor = .clear
panel.isOpaque = false
panel.level = .floating // 悬浮层级
panel.hasShadow = false // 阴影会有矩形轮廓，必须关
panel.collectionBehavior = [.canJoinAllSpaces, .fullScreenAuxiliary, .stationary]
panel.isFloatingPanel = true
panel.becomesKeyOnlyIfNeeded = true
```

逐个解释：

属性	为什么
.borderless	去掉标题栏/关闭按钮，窗口只剩"内容"
.nonactivatingPanel	点宠物时菜单栏仍然是当前 app 的——不然点一下猫，你的代码编辑器就"失焦"了
.clear 背景	窗口本身透明，只有画上去的猫可见
.floating	层级比普通窗口高，永远看得见
.canJoinAllSpaces + .stationary	切虚拟桌面/全屏时宠物不动、跟着走

2. 窗口布局：上下分区

窗口 200×240pt，下方正方形是宠物区，上方留给气泡：

```
private var petArea: NSRect {
let side = min(bounds.width, bounds.height)
return NSRect(x: 0, y: 0, width: bounds.width, height: side)
}
// 视图坐标系原点在左下：y=0 是底部
```

```
┌──────────────────┐ 240
│ 倒计时气泡 │ ← drawBubble()
│ 饥饿气泡 │ ← drawHungerBubble()（叠在倒计时上方）
├──────────────────┤
│ │
│ 猫咪 │ ← petArea（200×200）
│ │
└──────────────────┘
```

3. 像素级透明：draw(_:) 的第一行

```
override func draw(_ dirtyRect: NSRect) {
NSGraphicsContext.current?.cgContext.clear(dirtyRect) // 先清空成透明
// ... 再画帧、气泡、爱心
}
```

`clear` 把指定区域重置为全透明。**不清空会怎样？** macOS 视图的默认行为是"保留上次绘制内容"，移动窗口或重绘时会留下上一帧的残影——这是我们项目早期最著名的 bug「拖影」的元凶之一。

4. 为什么不用 layer-backed 视图

PetView 的 init 里有一行注释值得记住：

```
// 刻意不用 wantsLayer：layer-backed 视图在透明窗口里高频重绘会有残影
```

Cocoa 视图默认是"直接绘制到窗口 backing store"；开 layer 后走 Core Animation 事务合成，在透明窗口 + 高频重绘的场景下会留下中间态残影。这是个"踩过才知道"的坑，记住结论即可：**透明窗口上的逐帧动画，保持非 layer 视图**。

5. 自适应缩放：一个公式适配所有窗口尺寸

宠物大小可调（50%~150%），窗口尺寸 = 200s × 240s。视图内部不硬编码任何坐标，全部用 `bounds` 和 `viewScale` 推导：

```
func setScale(_ scale: CGFloat) {
let s = min(max(scale, 0.5), 1.5)
petView.viewScale = s
let newSize = NSSize(width: 200 * s, height: 240 * s)
// 保持窗口中心点不变缩放（不是左上角锚定，宠物不会"跳"）
let center = NSPoint(x: panel.frame.midX, y: panel.frame.midY)
panel.setFrame(NSRect(x: center.x - newSize.width / 2, ...), display: true, animate: false)
petView.frame = NSRect(origin: .zero, size: newSize)
}
```

视图里的气泡字号、内边距、尾巴偏移全部乘 `viewScale`——这就是"一处缩放、处处跟随"的写法。

6. 多屏与可见区域

乱窜和庆典移动都要知道"能飞多远"：

```
let visible = (panel.screen ?? NSScreen.main)?.visibleFrame
// visibleFrame = 屏幕减去 Dock 和菜单栏后的安全区域
```

注意用 `visibleFrame` 而不是 `frame`——否则宠物可能躲进 Dock 后面。

7. 验收清单：怎么验证一个透明悬浮窗做对了

测试	标准
显示	只有猫可见，无矩形背景
点击	猫响应，前台 app 不切换
全屏看视频	猫还在（不消失、不闪）
切 Space	猫留在原地
拖动	贴手，无延迟
明暗两种桌面壁纸	都无残影

下一篇讲让猫动起来的核心：序列帧动画引擎。

05 · 动画引擎：序列帧与省电重绘

FocusPet 零基础开发教程 · 第 5 篇
对应源码：PetView.swift 全文（重点）

1. 帧动画的本质

所谓动画，就是**按时间顺序快速切换图片**。电影里是 24 帧/秒，我们的宠物：

```
struct AnimationSpec {
let prefix: String // 素材前缀，如 "walk" / "dog_walk"
let count: Int // 帧数（我们统一 8 帧）
let frameDuration: CFTimeInterval // 每帧停留秒数
let holdLastExtra: CFTimeInterval // 每轮最后一帧额外停留（制造节奏感）
let repeatCount: Int? // nil=无限循环；数字=播完停在最后一帧
}
```

猫走路 0.6s/帧 → 慢悠悠；狗跑步 0.14s/帧 →
飞奔。**换一个数字就换一种性格**，这是调动画手感的第一旋钮。

2. 播放状态机

PetView 内部维护一套极简状态：

```
private var frames: [NSImage] = [] // 当前动画的帧数组
private var frameIndex = 0 // 现在该画第几帧
private var frameStartTime: CFTimeInterval // 当前帧是几点开始播的
private var finished = false // 有限循环播完了吗
```

推进逻辑（tick）：

```
if now - frameStartTime >= frameDurations[frameIndex] {
frameIndex += 1
frameStartTime = now
if frameIndex >= frames.count {
loopsDone += 1
if 播够了 { finished = true; 回调 completion() }
else { frameIndex = 0 } // 回到第一帧，再来一轮
}
}
```

核心思想：**不阻塞线程等待**，而是每次 tick
时问一句"该换帧了吗"。这叫"时间驱动"而非"延时驱动"，暂停、调速、跳帧都好做。

3. CVDisplayLink：跟着屏幕心跳走

```
屏幕每刷新一次（60Hz/120Hz）
→ CVDisplayLink 回调（在独立线程，极其精准）
→ 我们 dispatch 到主线程
→ 主线程执行 tick(now)
→ tick 里决定换不换帧、要不要重绘
```

为什么不用 `Timer`？Timer 受 RunLoop 模式影响（弹菜单时 Default 模式被切换，定时器会暂停），而 CVDisplayLink 跟着屏幕硬件走，**按屏幕实际刷新率回调**，精准且不停摆。

4. 素材加载：缓存是第一要务

```
func play(_ spec: AnimationSpec, ...) {
if frameCache[spec.prefix] == nil { // 首次播放才加载
frameCache[spec.prefix] = (1...spec.count).compactMap { index in
guard let url = Bundle.module.url(
forResource: "\(spec.prefix)_\(index)",
withExtension: "png",
subdirectory: "PetAssets"
) else { print(" 缺少素材"); return nil }
return NSImage(contentsOf: url)
}
}
frames = frameCache[spec.prefix] ?? []
...
}
```

128 张 PNG 全加载约百来 MB 内存，8 套动画按需加载 + 永久缓存（宠物 app 跑一整天也不会换几套），是内存和速度的最优平衡。

5. 绘制一帧

```
override func draw(_ dirtyRect: NSRect) {
NSGraphicsContext.current?.cgContext.clear(dirtyRect)
let area = petArea
drawFrame(frames[index], in: contentRect(for: frames[index], in: area))
drawHearts(...); drawBubble(); drawHungerBubble()
}
```

`contentRect` 做的事：按区域等比缩放图片并居中，再叠加弹跳缩放系数：

```
let scale = bounceScale * min(
(area.width - padding * 2) / image.size.width,
(area.height - padding * 2) / image.size.height
)
```

水平镜像（向右走转身）用 CGContext 的 scale(-1,1) 实现，**不能用 flipped**（和 NSView 自带属性冲突）。

6. 跨帧过渡：重影问题的来龙去脉

帧与帧直接硬切，快速动画会显得"跳"。加 crossfade（两帧叠加渐隐）更顺滑，但会多画一张图；且如果**两帧素材没有像素级对齐**，叠加时轮廓会糊成"重影"。

我们的对策（素材层解决，见第 12 篇）：

1. 所有帧用**同一个裁剪框**（8 帧内容边界的并集），保证角色在每帧中的相对位置严格一致
2. 过渡时长动态计算：`min(0.08s, 帧时长×0.35)`，帧越快过渡越短
3. 视图保持非 layer-backed（第 4 篇）

7. 省电重绘：本项目的第二次大优化

问题：CVDisplayLink 跟着 ProMotion 屏幕跑 120Hz，意味着每秒 120 次回调 + 120 次整窗重绘——哪怕宠物站着一动不动，GPU/CPU 也在空转。

三层防线（当前代码）：

```
// 第一层：回调里就过滤——没到换帧时刻，连主线程都不打扰
guard view.hasPendingWork(at: now) else { return kCVReturnSuccess }

// 第二层：tick 里比对——帧序号没变就不发 needsDisplay
if frameIndex != drawnFrameIndex {
drawnFrameIndex = frameIndex
needsDisplay = true
}

// 第三层：彻底静止就停表——动画播完、无弹跳、无爱心 → 停掉 DisplayLink
if !hasMotion { stopDisplayLink() }
// 恢复：play()/bounce()/showHearts() 里 startDisplayLink()
```

效果：待机时重绘频率从 120 次/秒降到 **3~4 次/秒**，静止时归零。

通用心法：动画系统的功耗 ∝ 重绘次数 × 重绘面积。优化永远是先减次数，再减面积，最后才考虑算法。

8. 爱心粒子系统（ mini 版粒子引擎）

双击时的爱心是"状态列表 + 每帧根据时间算位置"：

```
struct Heart { var x, y: CGFloat; var t0: CFTimeInterval; var emoji: String; ... }

// 绘制时（不是存储时）算位置：
let p = age / heartLife // 0→1 进度
let y = heart.y + p * 50 * viewScale // 上升
let alpha = 1 - p * p // 渐隐
// 画完自动清理过期粒子
```

粒子系统的通用范式就是这三步：**发射（append）→ 推进（按时间算状态）→ 回收（removeAll）**。

练习

1. 把猫 idle 的 `frameDuration` 从 0.28 改成 0.5，观察"变慢半拍"的慵懒感
2. 把 `holdLastExtra` 改成 0，观察节奏感消失
3. 注释掉省电第二层（needsDisplay 守卫），用活动监视器观察 CPU 差异

06 · 行为调度：让宠物「活」起来

FocusPet 零基础开发教程 · 第 6 篇
对应源码：PetController.swift 的行为调度部分

1. 目标

宠物不能只是循环播放待机动画——那是个屏保。我们要它**自己决定接下来干嘛**。拆解一下"活着"的感觉：

- 大多数时候待着（idle）
- 偶尔起来走两步（散步 stroll）
- 偶尔突然疯跑（乱窜 dash）
- 有事时（专注/被拖拽）停下一切
- 饿了就讨食

全部是**「随机数决定做什么 + 定时器决定何时做」**。没有人工智能，但效果以假乱真。

2. 调度器的通用骨架

项目里有两类调度，结构一模一样：

```
// 类型一：随机小动作（DispatchWorkItem 版）
private func scheduleRandomBehavior() {
idleWorkItem?.cancel() // ① 取消旧的
let delay = Double.random(in: 5...12) // ② 随机等待
let item = DispatchWorkItem { [weak self] in
self?.performRandomBehavior() // ③ 到点执行
}
DispatchQueue.main.asyncAfter(deadline: .now() + delay, execute: item)
idleWorkItem = item // ④ 记住它，方便取消
}

// 类型二：乱窜（Timer 版，逻辑相同）
private func scheduleRoam() {
roamTimer?.invalidate()
roamTimer = Timer.scheduledTimer(withTimeInterval: Double.random(in: 12...30),
repeats: false) { [weak self] _ in
self?.performRoam()
}
}
```

记住这个四步骨架：取消旧 → 随机延迟 → 执行 →
存引用。它是所有"周期性随机行为"的模板（讨食调度也是照抄的）。

3. 行为决策：掷骰子

```
switch Int.random(in: 0..<10) {
case 0...4: performStroll() // 50% 散步（最常见，闲不住）
case 5...6: play(.wave) {...} // 20% 挥手
case 7...8: play(.sleep) {...} // 20% 趴下睡会儿
default: break // 10% 继续待着
}
```

调概率就是调性格。想让它更粘人？把 wave 的概率调高。这就是"游戏手感调校"。

4. 移动：一次完整的散步是怎么实现的

`startMove(.stroll)` 的核心参数（猫 vs 狗）：

参数	猫	狗	效果
速度	55~95 px/s	130~190 px/s	猫遛弯，狗赶路
帧时长	0.6s	0.26s	猫抬腿慢，狗倒腾快
抛物线高度	≤14px	≤60px	狗明显在蹦

插值移动（每 16ms tick 一次）：

```
let raw = (CACurrentMediaTime() - roamT0) / roamDur // 0→1 进度
let e = raw < 0.5 ? 2 * raw * raw : 1 - pow(-2 * raw + 2, 2) / 2 // easeInOut
panel.setFrameOrigin(NSPoint(
x: from.x + (to.x - from.x) * e,
y: from.y + (to.y - from.y) * e + roamArc * sin(.pi * raw) // 抛物线
))
```

两个数学小动作值得记住：

1. **easeInOut**：起步慢、中间快、停稳慢。直接线性移动（`x = from + (to-from)*raw`）会像机器人滑轨；$2t^2$ / $1-(-2t+2)^2/2$ 这两段二次函数让移动有"重量感"
2. `sin(π · raw)`：0→1 之间正好是「0 升到 1 再落回 0」的光滑弧线，加在 y 上就是蹦跳

到达处理：

```
if raw >= 1 {
panel.setFrameOrigin(to) // 精确落点（防浮点误差）
timer.invalidate()
if 有庆典回调 { 执行庆典 } else { 回 idle }
scheduleRoam() // 排下一次乱窜
}
```

5. 打断与礼让：多调度器共存不打架

三个调度器可能同时想要宠物：随机小动作、乱窜、讨食。规则：

```
guard roamAnim == nil else { return } // 移动中不启动新移动
guard !petView.isBeingDragged else { ... } // 被拖拽时全部避让
guard !isSessionActive() else { return } // 专注时暂停自由行为
```

会话状态是最高优先级：`onSessionStart` 一口气取消 idleWorkItem 和 roamTimer；`onSessionEnd` 再重新调度。这就像红绿灯——不是每个司机互相让，而是大家都听信号灯。

6. 庆典移动：复用的胜利

"专注结束跑到屏幕中央"和散步/乱窜本质是同一件事（从 A 到 B）。我们给移动系统加了一个到达回调插槽：

```
private var onArriveMove: (() -> Void)?

// 到达分支：
if let cb = self.onArriveMove {
self.onArriveMove = nil
cb() // 接庆祝
} else if !self.isSessionActive() {
self.play(.idle)
}
```

`celebrateAtScreenCenter` 只是"设好目标点 + 挂上回调"——移动代码零重复。这就是第 3 篇说的"在中间加一根线"。

7. 踩坑实录：那次庆典没跑起来

最初实现庆典时，设好了目标坐标和速度，却忘了启动移动 Timer——宠物原地庆祝，休息照常。冒烟测试靠日志（"跑去屏幕中央庆祝！距离 817pt"之后没有"休息开始"）抓住了它。

教训：跨组件的新流程，一定要写"端到端冒烟测试"——不是测函数，而是模拟真实用户把流程走一遍。

练习

1. 给散步加一种新风格"巡视"：速度更慢、垂直移动更多
2. 把乱窜概率调成 0，观察宠物"变文静"
3. 把 easeInOut 换成线性插值，感受"机器人感"

07 · 交互系统：点击、拖拽、双击

FocusPet 零基础开发教程 · 第 7 篇
对应源码：Sources/FocusPet/PetView.swift（第 117–131、425–467 行）

上一篇讲完了宠物自己怎么动，这一篇讲**你怎么摆弄它**。本章要实现的交互有四个：

交互	触发方式	效果
拖拽	按住宠物拖动	宠物跟着鼠标走，被抓起时是 wave 表情
单击	点一下	投喂（后来加的喂食功能）
双击	快速点两下	庆祝 + 爱心飘出
右键	点鼠标右键	弹出系统菜单

听起来简单？这一章是**踩坑密度最高**的一章，三个经典的 mac 交互陷阱都会遇到。我们边写边踩。

一、mac 鼠标事件的来龙去脉

当你在 mac 上按一下鼠标左键，系统会产生一连串事件，按时间顺序进入你的窗口：

```
mouseDown(按下) → mouseDragged(拖动，可有多个) → mouseUp(抬起)
```

你的 NSView 子类可以通过覆写这些方法来响应。测试一下，在 PetView 里加一个临时打印：

```
override func mouseDown(with event: NSEvent) {
print("按下了！位置: \(event.locationInWindow)")
super.mouseDown(with: event)
}
```

event.locationInWindow 是这次点击在窗口坐标系里的位置——以后写拖拽都要靠它算偏移。

还有一个前置问题：无边框的 NSPanel 默认有个怪癖——**窗口非激活状态下，第一下点击只负责激活窗口，会被它吞掉**。用户盯着宠物发呆半天，第一次伸手去点，竟然没反应！解决办法是一行：

```
/// 无边框面板里的第一下点击也要能被接住
override func acceptsFirstMouse(for event: NSEvent?) -> Bool { true }
```

这行的意思是：「别管窗口激不激活，第一次点击直接给我」。

二、拖拽：performDrag 的零延迟魔法

2.1 第一版（错误示范）：跟着事件走

直觉的拖拽写法是在 mouseDragged 里不停地把窗口挪过去：

```
// 反例：别学这个
override func mouseDragged(with event: NSEvent) {
let newOrigin = ... // 用当前位置 + 本次位移算出窗口新位置
window?.setFrameOrigin(newOrigin)
}
```

这个写法功能上没问题，但**手感很差**：拖动时宠物会有一种「跟不上鼠标」的粘滞感。原因在源码注释里写得很清楚：

> 那条路径走事件队列，主线程被动画绘制占住时会跟不上光标（宠物被甩在鼠标后面）。

事件队列的完整链路是：硬件产生移动 → WindowServer 派发事件 → 事件进入队列等待 → 主线程从队列里取事件 → 调用你的 mouseDragged。链路一长，动画线程一忙，宠物就落后鼠标半拍。

2.2 正确姿势：把窗口交给 WindowServer

mac 提供了一个专门的 API：

```
window?.performDrag(with: event)
```

它的语义是：「**从现在起，这个窗口的纹理直接由 WindowServer 跟随光标拖动**，不再经过任何事件队列」。WindowServer 是系统的窗口合成进程，它直接拿硬件光标的坐标，所以窗口跟随是**零延迟**的，任何应用代码都无法与之竞争。

这也是源码里的那段注释要刻进骨头的原因：

```
// 注意：不要再覆写 mouseDragged 手动 setFrameOrigin —— 那条路径走事件队列，
// 主线程被动画绘制占住时会跟不上光标（宠物被甩在鼠标后面）。
```

2.3 performDrag 的阻塞特性

performDrag 还有一个重要性质：**它会一直阻塞，直到鼠标抬起才返回**。这意味着你把按下、拖动、抬起三个阶段全包在一个函数里写：

```
override func mouseDown(with event: NSEvent) {
isBeingDragged = true
onDragPickup?() // 被抓起：切表情
let originBefore = window?.frame.origin
window?.performDrag(with: event) // ← 卡在这里，鼠标抬起才往下走
onDragPutDown?() // 松手落地
isBeingDragged = false
// ……这里还能判断「拖了多远」
}
```

这种阻塞式 API 在 Swift 里不常见（我们习惯了回调和 async/await），但它在「按下-拖动-抬起」这种天然有边界的事件流里非常好用——逻辑天然按时间顺序排好，不用维护任何状态。

「被抓起时切表情」是用一个回调挂上去的（PetView 自己不决定玩什么动画，把决定权交给 PetController）：

```
/// 被抓起 / 被放下（由 PetController 接上对应表情）
var onDragPickup: (() -> Void)?
var onDragPutDown: (() -> Void)?
```

而 `isBeingDragged` 标志位则是给第 06 篇的乱窜调度器看的：宠物正在被你拎着，不能一边被拎一边乱跑，所以调度器每轮决策前都要检查这个标志。

三、单击与双击的战争

3.1 冲突在哪

单击是投喂，双击是庆祝。问题是：**双击在物理上就是两次单击**。系统怎么区分你的意图？

mac 的做法是：按下后等待一个「双击判定间隔」（系统设置里可调，默认约 0.25 秒），如果这段时间内没有第二次按下，才算单击；否则两次一起算双击。

这给程序员带来一个两难：

- 如果按下就立刻触发单击 → 用户双击时会被投喂两次，且第一次投喂的动画会干扰第二次
- 如果永远等双击判定 → 单击的响应被拖慢 0.25 秒，手感迟钝

我们的解法是**延迟确认**：

```
/// 单击判定窗口：按下到抬起间隔太短会被双击判定点干扰，延迟确认
private var tapWorkItem: DispatchWorkItem?
```

3.2 clickCount：系统已经帮你数好了

`NSEvent` 自带一个 `clickCount` 属性：第一次按下它是 1，双击的第二次按下它是 2。所以在 `mouseDown` 里一行就能分流：

```
override func mouseDown(with event: NSEvent) {
// 双击：触发互动，不进入拖拽；同时取消刚排队的单击（避免双击变两次投喂）
if event.clickCount >= 2 {
tapWorkItem?.cancel() // 把还在排队等确认的单击取消掉！
tapWorkItem = nil
onDoubleTap?()
return
}
// ……单击/拖拽逻辑
}
```

那句 tapWorkItem?.cancel() 是整个防误触的关键：**双击发生时，第一次按下时排队的那个「延迟确认单击」任务还在调度队列里等着执行，必须先取消它**，不然就会「双击 = 一次投喂 + 一次庆祝」。

3.3 DispatchWorkItem：可以取消的定时任务

确认单击用的是 GCD 的延迟执行：

```
// 原地单击（几乎没拖动）→ 延迟确认投喂，给双击判定留窗口
if let before = originBefore, let after = window?.frame.origin,
hypot(after.x - before.x, after.y - before.y) < 6 {
tapWorkItem?.cancel()
let item = DispatchWorkItem { [weak self] in
guard let self, !self.isBeingDragged else { return }
self.onTap?()
}
DispatchQueue.main.asyncAfter(deadline: .now() + 0.28, execute: item)
tapWorkItem = item
} else {
tapWorkItem?.cancel()
tapWorkItem = nil
}
```

逐行拆：

代码	作用
hypot(after.x - before.x, after.y - before.y) < 6	用位移距离区分「单击」和「拖了一下」：抬起时如果挪了不到 6 个点，算单击
DispatchWorkItem { ... }	把一段代码包成一个「可取消的任务对象」
DispatchQueue.main.asyncAfter(deadline: .now() + 0.28, execute: item)	0.28 秒后**在主线程**执行这个任务
tapWorkItem?.cancel()	任务还没执行就取消它——这就是双击救场的按钮

为什么选 0.28 秒？系统的双击判定间隔通常在 0.25 秒左右，留 0.28 秒让第二次按下来得及取消第一次排队的任务；又不会让用户等太久，单击手感依然轻快。

工作项里的双重保险也值得一提：

```
guard let self, !self.isBeingDragged else { return }
```

[weak self] 防止视图被销毁后任务还强引用着它（内存泄漏的常规防御）；!self.isBeingDragged 则是一个边角情况：理论上 0.28 秒内用户不可能既拖完又等任务执行，但加了这层判断，即使发生了也不会在拖拽中途误投喂。

注意一个顺序细节：performDrag 返回之后我们才做单击判定——因为距离（before/after）只有等拖完才知道。整个

`mouseDown` 方法把按下、拖动、抬起、延迟判定串成了教科书级的顺序逻辑，全靠 `performDrag` 的阻塞特性。

四、右键

右键反而最简单，一行回调透传出去：

```
override func rightMouseDown(with event: NSEvent) {
onShowMenu?(event)
}
```

为什么是回调而不是直接在这里建菜单？因为「菜单里有什么」是应用层的事——设置入口、切换宠物、退出——这些属于 AppDelegate 的职责。PetView 只负责「有人右键了」这个事实。这种「视图报事件、上层做决定」的分层，在整套项目里反复出现（后面讲 FocusEngine 回调时你会再次见到）。

五、四种交互的全景

把四种交互的判定方式排在一起，这张表值得收藏：

交互	判定依据	关键 API	踩过的坑
拖拽	按下后位移超过 6 点	`window.performDrag`	手动 `setFrameOrigin` 走事件队列会卡顿
单击	位移 < 6 且 0.28 秒内无第二击	`DispatchWorkItem` + `asyncAfter`	不延迟确认会被双击误触发
双击	`event.clickCount >= 2`	取消 `tapWorkItem`	双击前必须先取消排队中的单击
右键	`rightMouseDown` 回调	`onShowMenu?()`	—

加上 `acceptsFirstMouse` 和 `isBeingDragged` 这两个辅助件，整个交互系统的状态流转如下：

```
mouseDown
 ├── clickCount ⩾ 2 → 取消单击任务 → onDoubleTap（爱心庆祝）
 └── clickCount = 1 → isBeingDragged = true → onDragPickup（被抓起表情）
↓
performDrag（窗口零延迟跟手，调度器避让）
↓
抬起 → onDragPutDown → isBeingDragged = false
 ├── 位移 < 6 → 0.28s 后确认 → onTap（投喂）
 └── 位移 ⩾ 6 → 什么都不做，就是拖了个位置
```

练习

1. 把确认单击的延迟从 0.28 改成 0.15 秒，多双击几次，感受误触率的变化；再改成 0.5 秒，感受单击的迟钝。
2. 在 `mouseDown` 里加一句 `print(event.clickCount)`，观察单击、双击、三击时这个数字各是多少。
3. 尝试给宠物加「三击彩蛋」：三击时切换成 sleep 动画。提示：`clickCount` 会一直往上数。
4. 思考题：为什么拖拽判定的阈值是 6 而不是 0？如果设成 0，鼠标最轻微的一抖会发生什么？（答案和「双击判定」有关。）

08 · 番茄钟引擎：状态机思维

FocusPet 零基础开发教程 · 第 8 篇
对应源码：Sources/FocusPet/FocusEngine.swift（共 141 行，本篇全部内容都在这一个文件里）

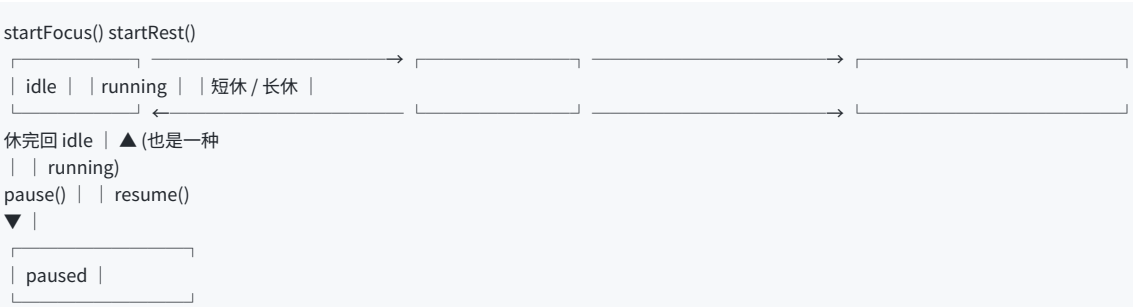
这是整个 App 的「灵魂功能」——25 分钟专注倒计时。你可能预期它需要几百行代码，实际上**核心逻辑只有 141 行**。这得益于一个威力巨大的思想工具：**状态机**。本章就教你用它。

一、什么是状态机

状态机（State Machine）一点也不玄，它回答的问题就一句话：

> **「现在处于什么状态？每个状态允许做哪些事？」**

拿电梯想：电梯只有「开门、关门、上行、下行、停止」几个状态，每个状态下只接受特定的操作——门开着的时候不可能接受「上行」指令。番茄钟也一样，它的状态图是一张有限的小地图：



对照 FocusEngine 里的枚举，一字不差：

```
enum Phase {
case idle // 空闲
case running // 倒计时中
case paused // 暂停（冻住剩余时间）
}
```

还有会话类型的枚举：

```
enum Kind: String {
case focus
case shortRest
case longRest

var isRest: Bool { self != .focus }
}
```

设计状态机时最重要的一步是**穷举状态**和**规定转移**。看 pause() 的第一行：

```
func pause() {
guard phase == .running else { return } // 只有 running 才能暂停
setPhase(.paused)
}
```

`guard` 做「非法操作直接拒绝」——空闲时按快捷键，什么都不会发生，而不是冒出奇怪行为。状态机的价值就体现在这里：**它让每个非法操作都有明确的归宿（被忽略），而不是产生未定义行为**。

经典番茄钟规则：每完成 4 个专注，来一次 15 分钟长休，其余都是 5 分钟短休。这个规则在 `startRest()` 里就一行：

```
/// 按「已完成专注数」决定短休还是长休（每 focusesPerCycle 轮一次长休）
func startRest() {
let isLong = completedFocuses > 0 && completedFocuses% focusesPerCycle == 0
start(session: Session(
kind: isLong ? .longRest : .shortRest,
duration: isLong ? longRestDuration : shortRestDuration
))
}
```

`%` 是取余运算符：`completedFocuses% 4 == 0` 意为「完成的专注数是 4 的倍数」。第 01 篇语法篇讲过它，这里就是它的实战。

二、回调：纯逻辑引擎与 UI 的分界线

打开 `FocusEngine.swift` 看一眼——这个文件**从头到尾没有 import AppKit，没有一行 UI 代码**。这是刻意的设计：

> 纯逻辑不碰 UI。

引擎只负责计算，怎么把结果呈现出来（气泡里显示 24:59？状态栏图标变红？宠物切 focus 动画？）它一概不管。双方靠四个回调沟通：

```
/// 每秒回调一次剩余秒数
var onTick: ((TimeInterval) -> Void)?
/// 运行/暂停/空闲 切换
var onPhaseChange: ((Phase) -> Void)?
/// 新会话开始
var onSessionStart: ((Session) -> Void)?
/// 会话自然走完（专注/休息完成）
var onSessionComplete: ((Session) -> Void)?
```

回调就是「留个电话号码给我，有情况我打给你」。AppDelegate 在组装时把号码填上：

```
// 伪代码，展示接线方式
engine.onTick = { remaining in
petView.updateTime(format(remaining)) // 气泡显示倒计时
statusBar.updateTitle(" \(format(remaining))")
}
engine.onSessionComplete = { session in
// 宠物跑屏幕中央庆祝 + 弹系统通知 + 写统计记录
}
```

注意回调都是 `Optional`（类型后面带 `?`），调用时要写问号：

```
self.onTick?(self.remaining)
```

含义是「号码留了才打，没留就跳过」。这带来一个巨大的好处：**FocusEngine 可以在没有任何 UI 的单元测试里独立运行**——这是专业代码与玩具代码的分水岭之一。

这个模式在项目中反复出现，给它起个名字记牢：**好莱坞原则**（"Don't call us, we'll call you"）——引擎不主动去找视图，而是视图把回调留给引擎。

三、Timer：常驻 + 闸控，防漂移的巧思

3.1 两种暂停设计

倒计时每秒走一格，朴素的想法是「暂停就把 Timer 销毁，恢复再建一个」：

```
// 朴素但有隐患的写法
func pause() {
timer?.invalidate() // 销毁
timer = nil
}
```

隐患在哪？`Timer.scheduledTimer(withTimeInterval: 1, ...)` 的触发时刻是相对「创建时刻」的。销毁再重建，新 Timer 的第一秒是从「创建」开始算，而不是从「恢复」那一瞬间精确对齐，多次暂停恢复后，**倒计时会慢慢漂移**（比如 25 分钟的专注实际过了 25 分 07 秒）。

3.2 我们的方案：Timer 一直活着，闸门控制

看 `start(session:)` 里 Timer 的闭包：

```
// 暂停时不倒计时，所以 timer 常驻、闸控即可，避免暂停/恢复计时漂移
timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
guard let self, self.phase == .running else { return } // ← 闸门
self.remaining = max(0, self.remaining - 1)
self.onTick?(self.remaining)
if self.remaining <= 0 {
self.completeSession()
}
}
```

每秒 Timer 照常在响，但闭包第一行先检查 `phase == .running`——**暂停期间，Timer 响归响，剩余时间纹丝不动**。恢复时也不需要重建任何对象，下一秒的滴答自然继续。Timer 的触发节拍完全不受暂停影响，漂移问题被根除。

这个技巧叫**「常驻 + 闸控」（always-on with gating）**，在工程上非常常见：让节奏源保持稳定，用状态标志位决定「这一拍要不要生效」。

闭包里还有两个细节：

- `[weak self]`：又是它。Timer 会强引用这个闭包，闭包如果强引用 self（引擎），引擎就永远不能被释放——循环引用。`weak` 打破这个环。`guard let self` 把弱引用变回强引用供本次使用。
- `max(0, self.remaining - 1)`：防御性编程，保证剩余时间永不为负（虽然后面的 `<= 0` 判断已经兜底，双保险不亏）。

3.3 完成一次会话

```
private func completeSession() {
stopTimer()
guard let finished = session else { return }
if finished.kind == .focus {
completedFocuses += 1 // 只有「专注」走完才计数，休息不算
}
session = nil
remaining = 0
setPhase(.idle)
onSessionComplete?(finished)
}
```

这里藏着一条产品规则：**只有专注走完才计入 completedFocuses**（中断放弃的不算，看 `stop()` 就知道——它直接清场，不动计数器）。所以那个「每 4 次长休」的规则天然只统计「货真价实完成的专注」。

四、setPhase 的小细节：只在变化时通知

```
private func setPhase(_ newPhase: Phase) {
guard phase != newPhase else { return } // 状态没变就不通知
phase = newPhase
onPhaseChange?(newPhase)
}
```

为什么要多这层判断？假想没有它：空闲时用户连按五次快捷键，`pause()` 的 guard 会挡住非法转移，但 `stop()` 这类方法每次都调用 `setPhase(.idle)`——没有这层判断，就会重复触发五次 `onPhaseChange`，上层收到五次「回到空闲」的通知，菜单重建五次。**「只在变化时通知」是观察者模式的通用礼仪。**

枚举的相等比较（`phase != newPhase`）之所以能直接用 `==`，是因为 Swift 的枚举自动获得相等性——不需要你手写比较逻辑。

五、配置即属性

引擎的四个参数是普通属性，随时可改：

```
var focusDuration: TimeInterval // 默认 25 * 60 秒
var shortRestDuration: TimeInterval // 默认 5 * 60
var longRestDuration: TimeInterval // 默认 15 * 60
var focusesPerCycle: Int // 默认 4
```

用户在设置面板里把专注改成 45 分钟，改的就是这里的 `focusDuration`。下一次 `startFocus()` 自然生效，不用重启、不用刷新——因为 `start(session:)` 每次都用**当前**配置值新建会话。

默认值在构造函数里给：

```
init(
focusDuration: TimeInterval = 25 * 60,
...
) {
self.focusDuration = focusDuration
...
}
```

`TimeInterval` 就是 `Double` 的别名（秒数），`25 * 60` 算出 1500 秒。用乘法写比直接写 1500 的可读性好得多——读者一眼知道是「25 分钟」。

六、把 141 行装进脑子

最后用一个表格总结 FocusEngine 的公共 API，这就是它的全部对外面孔：

方法/属性	状态要求	干什么
`startFocus()`	任意（会停掉旧会话）	开始一段专注
`startRest()`	任意	按完成数自动选短休/长休
`pause()`	running	暂停（闸控，不杀 Timer）
`resume()`	paused	继续
`stop()`	任意	提前结束，不计完成数
`onTick / onPhaseChange / onSessionStart / onSessionComplete`	—	四个回调插槽

回想第 03 篇的类分工地图：FocusEngine 是这个项目里「纯度」最高的一个类——**没有 UI 依赖、没有单例、状态全靠自己、靠回调对外**。以后你自己写任何「计时/流程」类功能（比如闹钟、游戏回合、下载重试），都可以照抄这个骨架：枚举定义状态 → 方法控制转移（guard 拦截非法转移）→ 回调通知外界。

练习

1. 在 Playground 里创建一个 FocusEngine，接上四个回调并全部 print，然后按这个顺序操作：`startFocus → pause → resume → stop`，观察回调序列和 phase 变化。
2. 把「每 4 次长休」改成「每 3 次」，只改调用处传入的参数即可——体会「配置即属性」的好处。
3. 思考题：`stop()` 里为什么调用 `setPhase(.idle)` 而不是直接 `phase = .idle`？（提示：看看 setPhase 里还干了什么。）
4. 进阶：给引擎加一个 `skipRest()`（跳过当前休息直接回 idle），它和你想象中的实现有差别吗？需要动回调吗？

09 · 状态栏图标、全局快捷键与系统通知

FocusPet 零基础开发教程 · 第 9 篇
对应源码：Sources/FocusPet/StatusBarController.swift（36 行）、Sources/FocusPet/HotkeyManager.swift（70 行）、Sources/FocusPet/AppDelegate.swift（通知与热键接线部分）

一个没有 Dock 图标、没有主窗口的菜单栏应用，和用户沟通的三个通道是：状态栏图标、全局快捷键、系统通知。本章把这三个小零件一次讲完——它们每一个都藏着至少一个真实的坑。

一、状态栏图标

1.1 最小骨架

mac 的状态栏图标官方叫 **Status Item**。最小的代码三行：

```
let statusItem = NSStatusBar.system.statusItem(withLength: NSStatusItem.variableLength)
statusItem.button?.title = "" // 用 emoji 当图标
statusItem.button?.action = #selector(handleClick)
```

FocusPet 里把它包成了一个 36 行的类（StatusBarController）。第一行值得细看：

```
private let statusItem = NSStatusBar.system.statusItem(withLength: NSStatusItem.variableLength)
```

- NSStatusBar.system：系统状态栏的单例。
- statusItem(withLength:)：申请一个位置。variableLength 表示「宽度随内容自适应」——因为我们要在图标旁边显示倒计时文字（24:59），宽度会变。如果内容固定可以传一个具体点数。
- 申请到的是一个 NSStatusItem，它内部有个 button（就是你在菜单栏看到的那个可点区域）。

为什么 button 后面要跟问号？因为理论上申请可能失败、button 可能为 nil。Swift 的可选链语法：a?.b 表示「a 不是 nil 才取 b，是 nil 就整句静默跳过」——这比满地 if-let 写起来轻快，适合这种「失败也无所谓」的赋值场景。

1.2 一个史诗级大坑：accessory 应用点图标没反应

这是本项目开发过程中真实踩过的坑，用户也反馈过「点击状态栏图标好像没啥用」。症状：点击图标，菜单偶尔弹出、偶尔没反应，弹出了也点不动里面的项。

根源藏在应用类型的选择里。FocusPet 是菜单栏应用，Info.plist 里设置了：

```
<key>LSUIElement</key><true/>
```

这让应用成为 **accessory（附属）应用**：不显示在 Dock，不出现在 Cmd+Tab 切换列表。副作用是——**这类应用默认永远不成为「活跃应用」**。而菜单弹出后要接收你的点击，系统要求「菜单所属的应用是活跃的」。矛盾就此产生：点了图标 → 菜单弹出 → 应用却不活跃 → 点击事件被拒之门外。

注释里记录了教训：

```
/// 关键：先激活应用（accessory 模式默认不激活），
/// 否则菜单弹出后可能收不到点击、设置/统计窗口也可能不显示
NSApp.activate()
menu.popUp(
positioning: nil,
at: NSPoint(x: 0, y: button.bounds.height),
in: button
)
```

`NSApp.activate()` 就是把应用临时拉回「活跃」状态。顺序不能反：先激活，再弹菜单。这个坑在 Stack Overflow 上常年霸榜，无数菜单栏应用作者都栽过。

1.3 为什么每次点击都重建菜单

注意这个回调的签名：

```
/// 点击时由 AppDelegate 重新构建菜单
var onRebuildMenu: (() -> NSMenu)?
```

点一次图标就构建一次菜单？看起来浪费，其实必要。因为菜单内容**时刻在变**：专注中菜单应显示「暂停/放弃」，空闲时应显示「开始专注」，倒计时数字也要实时。与其维护一套「增量更新菜单项」的复杂机制，不如每次点击时按当前状态从头构建一份——构建一份菜单的开销微乎其微，换来的却是「菜单永远和状态同步」的简单保证。

另外一行也值得说：

```
statusItem.button?.sendAction(on: [.leftMouseDown, .rightMouseDown])
```

默认只有鼠标抬起（up）才触发 action，改这行让它在**按下**瞬间触发——菜单弹出的响应更快，更有「跟手」感。

二、全局快捷键：按 Shift+Cmd+P 就能开始专注

2.1 为什么走 Carbon 而不是 SwiftUI

你可能听过 `KeyboardShortcuts` 之类的 Swift 库。我们用的是更底层的 **Carbon** 事件 API（`RegisterEventHotKey`），原因写在注释里：

```
/// 全局快捷键（Carbon RegisterEventKey，无需辅助功能权限）
```

mac
的权限模型里，「监听全局键盘」通常需要用户在系统设置里手动授予「辅助功能」权限；而 Carbon 这个古老的注册式 API 是个历史遗留的绿色通道——注册一个「按下组合键时给我发个事件」的热键，**不需要任何权限弹窗**。对一个桌面宠物来说，装完即用、不弹 scary
的权限对话框，体验完胜。

代价是 API 古老（C 风格），70 行里一大半是样板。我们直接看它最精华的注册调用：

```
let hotKeyID = EventHotKeyID(signature: OSType(0x4650_4554), id: 1) // 签名 'FPET'
let status = RegisterEventHotKey(
keyCode, // 键码：kVK_ANSI_P = 字母 P
modifiers, // 修饰键：cmdKey | shiftKey = Shift+Cmd+
hotKeyID, // 我们自己定义的热键 ID（收到事件时用来认领）
GetApplicationEventTarget(), // 事件投递目标：本应用
0,
&hotKeyRef // 输出：注册成功后拿到一个句柄，注销时要用
)
```

三个概念：

概念	说明
keyCode	物理键的编号。kVK_ANSI_P 是 P 键。为什么不用字符而用编号？因为同一个字母在不同键盘布局上位置不同，编号才是物理事实
modifiers	修饰键掩码。`cmdKey
hotKeyRef	注册凭证。UnregisterEventHotKey(hotKeyRef) 注销时必须归还，所以它是可选类型成员，类销毁时（deinit）记得注销

2.2 Unmanaged：Swift 与 C 之间传递「我自己」

HotkeyManager 是 Swift 类，但 Carbon 的回调是 C 函数指针——它不懂 Swift 的引用计数，只认识一个 void* 指针。把一个 Swift 对象交给 C 世界的标准姿势是 Unmanaged：

```
let handler: EventHandlerUPP = { _, _, userData in
guard let userData else { return noErr }
let manager = Unmanaged<HotkeyManager>.fromOpaque(userData).takeUnretainedValue()
DispatchQueue.main.async {
manager.onTrigger?()
}
return noErr
}
```

- Unmanaged.passUnretained(self).toOpaque()（注册时）：「给 C 一个指向我的指针，但不增加我的引用计数」。为什么敢不增加？因为这个指针的生命期被我们约束在「handler 注册期间」——类销毁前先注销 handler，指针就不会悬空。

- `Unmanaged<HotKeyManager>.fromOpaque(userData).takeUnretainedValue()`（回调时）：「把 C 还回来的指针翻译回 Swift 对象」。

这套模式在 PetView 的 DisplayLink 回调里你已经见过一次（第 05 篇），这是 Swift 调底层 C API 的通用套路，记住这个形状就好。

另外一个细节：回调可能在任意线程被调用，而触发热键要做的事（读引擎状态、改 UI）都必须在主线程。所以回调体第一行永远是：

```
DispatchQueue.main.async { manager.onTrigger?() }
```

「回调回到主线程再碰 UI」是 GUI 编程的军规。

2.3 热键的智能：一个键三个动作

AppDelegate 里只注册了一个热键，却实现了三个动作，靠的是查状态机：

```
hotkey.onTrigger = { [weak self] in
guard let self else { return }
switch (self.engine.phase, self.engine.session?.kind) {
case (.idle, _):
self.startFocus()
case (.running, .focus?):
self.pauseSession()
case (.paused, .focus?):
self.resumeSession()
default:
break // 休息中：什么都不做
}
}
```

这是对第 08
篇状态机的直接应用：快捷键不做决定，它只查询状态机然后翻译成对应的引擎调用。`switch` 对元组（`phase` 和 `kind` 的组合）做模式匹配——Swift 的模式匹配能一次拆两个值，`.focus?` 里的问号是在匹配「session 存在且 kind 是 focus」。

三、系统通知

3.1 现代 API：UserNotifications 框架

发一条系统通知的代码在 AppDelegate 的 `notify(title:body:)` 里，拆开看是四步：

```
let content = UNMutableNotificationContent()
content.title = title
content.body = body
content.sound = .default
let request = UNNotificationRequest(
identifier: UUID().uuidString, // 每次随机 ID → 通知永远不会被去重
content: content,
trigger: nil // nil = 立即发送
)
UNUserNotificationCenter.current().add(request)
```

`identifier` 用 UUID 是个实用小技巧：系统会用 identifier 去重「同 ID 的待展示通知」。如果固定一个 ID，连续完成两轮专注时，第二条通知会替换掉第一条还没看的。随机 ID 让每条通知都独立展示。

3.2 环境探测：裸可执行文件别碰通知

这一行是血泪换来的：

```
/// 系统通知只有在正式 .app bundle 中才可用（swift run / Xcode 直接跑的裸可执行文件
/// 调用 UNUserNotificationCenter 会直接崩溃）
private var notificationsAvailable: Bool {
Bundle.main.bundleURL.pathExtension == "app"
}
```

开发时你用 `swift run` 跑的是裸可执行文件，不是 `.app` 包。裸可执行文件调用通知中心会直接**崩溃**（不是报错，是 crash）。所以所有通知相关代码都裹在 `guard notificationsAvailable else { return }` 里。第 13 篇讲打包时会再解释 bundle 是什么——现在先记住：**判断「当前是不是正式 .app 环境」用 `Bundle.main.bundleURL.pathExtension == "app"`**。

首次启动还要请求授权：

```
UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .sound]) { granted, _ in
print("[FocusPet] 通知授权: \(granted ? "已允许" : "被拒绝")")
}
```

授权弹窗一生只会弹一次，之后用户在系统设置里改。回调里的 `granted` 告诉我们结果——但我们其实不需要据此改变行为：授权了通知能弹出，没授权就静默失败，应用本身照常运行。**「优雅降级」**：依赖的外部能力缺席时，核心功能不受影响。

四、三件套的分工总结

零件	类	核心技术	本章的坑
状态栏图标	StatusBarController	NSStatusItem	accessory 应用要先 NSApp.activate() 再弹菜单
全局热键	HotkeyManager	Carbon RegisterEventHotKey	C 回调用 Unmanaged 传 self；回调切主线程
系统通知	AppDelegate.notify	UserNotifications	裸可执行文件调用会崩溃，要探测 bundle

注意这三件套共同的**结构模式**：每个零件都是一个「小类 + 回调插槽」，自己不管业务，业务全在 AppDelegate 接线。StatusBarController 不知道菜单里有什么；HotkeyManager 不知道 Shift+Cmd+P 会触发什么；通知不知道自己在庆祝什么。这种「零件只管能力，装配决定行为」的风格，让任何一件都可以单独替换（比如以后想把 Carbon 热键换成 KeyboardShortcuts 库，AppDelegate 一行不动）。

练习

1. 把状态栏图标的 title 改成你自己的 emoji，再把 updateTitle 接到 onTick 上，让菜单栏实时显示倒计时。
2. 给热键换一个键（提示：查 kVK_ANSI_* 常量表），注意 register 可以反复调用——它内部会先 unregister 再注册。
3. 在 swift run 环境下故意调用 UNUserNotificationCenter，验证注释里说的「会直接崩溃」是不是真的（建议先在 Playground 里试）。
4. 思考题：为什么 HotkeyManager 的 deinit 里要注销热键？如果忘了注销，进程退出后会发生什么？

10 · 设置面板与数据持久化

FocusPet 零基础开发教程 · 第 10 篇
对应源码：Sources/FocusPet/SettingsModel.swift（123 行）、Sources/FocusPet/SettingsView.swift（180 行）

「设置」是所有应用躲不开的标配。它背后有两个经典问题：**数据放哪？（持久化）**和**界面怎么写？（设置面板）**。本章用 FocusPet 的方案把两个问题一次讲透。好消息是：设置面板是这个项目里**唯一用到 SwiftUI** 的地方——你在第 02 篇学过的 SwiftUI 知识（VStack、@State、$ 绑定）将全部派上用场。

一、持久化选型：UserDefaults

设置数据的特征很明确：量少（十几个值）、结构简单（数字、开关、字符串）、读写频繁。这种数据的标准归宿是 **UserDefaults**——mac 系统提供的轻量键值存储。

UserDefaults 的用法简单到不像话，就三个动作：

```
let defaults = UserDefaults.standard

defaults.set(45, forKey: "focusMinutes") // 写
let minutes = defaults.double(forKey: "focusMinutes") // 读
defaults.removeObject(forKey: "focusMinutes") // 删
```

它本质上是一张「字符串键 → 任意值」的字典，由系统自动落盘到 ~/Library/Preferences/ 下的一个 plist 文件。你不用管文件路径、不用管序列化、不用管什么时候写盘——系统替你全包了。

一个读写的类型陷阱

读的时候有个著名的坑。看 SettingsModel 初始化里的读法：

```
focusMinutes = defaults.object(forKey: Keys.focusMinutes) as? Double ?? 25
```

为什么不直接写 defaults.double(forKey:)？因为 double(forKey:) 在**键不存在时会返回 0**——你无法区分「用户存了 0」和「从来没存过」。而 object(forKey:) 返回可选值：键不存在就是 nil。于是 as? Double ?? 25 的完整语义是：

> 「如果存过且确实是 Double，用它；否则（含从来没存过的情况）用默认值 25。」

首次启动的应用没有任何历史设置，这行代码保证每个字段都有合理的出厂值。这就是「默认值兜底」模式，UserDefaults 编程天天用。

键名集中管理

```
private enum Keys {
static let focusMinutes = "focusMinutes"
static let shortRestMinutes = "shortRestMinutes"
...
}
```

把所有键名收进一个私有枚举，好处有二：一是避免手滑打错字符串（"foucsMinutes" 这种 typo 编译器可发现不了）；二是键清单一目了然。private 修饰保证只有 SettingsModel 内部能用这些键——**键的命名空间是模型私有的**，外界不应该绕过模型直接读写。

二、@Published + didSet：数据变动的双通道

SettingsModel 是一个 ObservableObject（第 02 篇的概念：「可被观察的对象」），它的每个属性都挂着两个装饰器：

```
@Published var focusMinutes: Double {
didSet { UserDefaults.standard.set(focusMinutes, forKey: Keys.focusMinutes); notify() }
}
```

这一行里塞了三层机制，层层都有明确职责：

机制	干什么
@Published	告诉 SwiftUI「这值变了请刷新界面」——数据 → 界面的通道
didSet（属性观察器）	每次赋值后自动执行：写入 UserDefaults（持久化）+ 调 notify()（通知 AppDelegate 即时生效）
notify() → onChange	让 AppDelegate 立刻把新值应用到引擎、宠物尺寸等——数据 → 业务逻辑的通道

也就是说，**一次赋值触发三条路**：界面刷新、磁盘落盘、业务生效。用户在设置面板里拖一下滑杆，三路人马同时开动，这就是设置窗口底部那句「所有设置即时生效」的技术底气。

三条路里两条是 Swift 内建机制（@Published、属性观察器），第三条（onChange 回调）是我们自己加的——因为有些设置 SwiftUI 管不到（比如 FocusEngine 的时长、状态栏热键的注册注销），必须靠 AppDelegate 手动应用。

Date? 的可选值持久化

日期类设置（生日、纪念日）是 Date?，存储时转成时间戳（Double）：

```
private func saveOptional(_ date: Date?, key: String) {
if let date {
UserDefaults.standard.set(date.timeIntervalSince1970, forKey: key)
} else {
UserDefaults.standard.removeObject(forKey: key)
}
}
```

`Date` 不能直接塞进 UserDefaults（实际上可以，但跨版本兼容性不如原始 Double 稳），而「从 1970 年到现在的秒数」这个 Double 存哪都安全。nil 则对应**删除键**——UserDefaults 里没有「null」概念，「没设过」和「设为空」是同一个状态。

三、设置面板：SwiftUI 的一次完整实战

打开 `SettingsView.swift`，从上到下能看到 SwiftUI 的常用控件全家桶：

3.1 声明式布局

```
var body: some View {
VStack(spacing: 16) {
GroupBox("番茄钟节奏") {
VStack(spacing: 10) {
Stepper("专注时长：\(Int(model.focusMinutes)) 分钟",
value: $model.focusMinutes, in: 5...120, step: 5)
...
```

- `VStack(spacing:)`：竖着排，子项间距 16 点。想要横排就换 `HStack`。
- `GroupBox("标题") { ... }`：mac 原生风格的分组卡片——标题在左上，内容包在圆角框里。设置类界面的标准件。
- `Stepper`：步进器，点 +/- 调数字。`value: $model.focusMinutes` 是双向绑定，`5...120` 是值域，`step: 5` 是步长。
- `some View`：返回类型占位，意思是「某种视图，具体哪种编译器自己推」。

标签里的 `\(Int(model.focusMinutes))` 是字符串插值——`model.focusMinutes` 是 Double（25.0），`Int(...)` 转成整数显示成「25 分钟」。

3.2 绑定：$ 号的一往一返

这是 SwiftUI 最核心的概念，值得再强调一次。`model.focusMinutes` 前面加 `$`：

```
$model.focusMinutes ←→ 一个 Binding<Double>
```

`Binding` 是个「读写双向通道」：控件读出它显示当前值；用户操作时通过它写回新值。写回时触发什么？——上一节的 `@Published` + `didSet` 三通道全开。所以从「用户拖滑杆」到「引擎拿到新时长」的完整链路是纯声明式的，**你没有写一行事件处理代码**。

3.3 一个宠物、两套资料：计算型绑定

项目里有个小麻烦：猫和狗各有名字和生日（共 4 个存储字段），但界面上一次只展示当前选中宠物的那套。滑杆切到「狗狗」就该编辑 `dogName`，切回「猫咪」编辑 `catName`。`TextField` 需要一个 `Binding<String>`，可它绑哪个？

解法是自己造一个「分流绑定」：

```
private var nameBinding: Binding<String> {
Binding(
get: { self.model.selectedPet == "dog" ? self.model.dogName : self.model.catName },
set: { newValue in
if self.model.selectedPet == "dog" {
self.model.dogName = newValue
} else {
self.model.catName = newValue
}
}
)
}
```

`Binding` 可以手动构造：给个 `get`（读时去哪取）和 `set`（写时存到哪）。控件无感知——它只认「这是个 Binding」，背后分流到猫还是狗它不关心。这是「**适配器模式**」的 SwiftUI 版：接口不变，内部按需转换。

3.4 可选日期：开关 + 选择器的组合

生日是可选的（`Date?`），但 SwiftUI 的 `DatePicker` 只接受非可选的 `Date`。`optionalDatePicker` 这个自定义组件把「有没有生日」翻译成界面上一个开关：

```
Toggle(label, isOn: Binding(
get: { date.wrappedValue != nil },
set: { date.wrappedValue = $0 ? Date() : nil } // 打开 = 填上今天；关闭 = 清空
))
if date.wrappedValue != nil {
DatePicker("", selection: Binding(
get: { date.wrappedValue ?? Date() },
set: { date.wrappedValue = $0 }
), displayedComponents: .date)
}
```

又一个手工 Binding：开关的「开/关」和日期的「有/无」互相翻译。开关打开时默认填「今天」（省得用户滚半天日期），出现日期选择器；关掉则清空并隐藏选择器。界面呈现和数据模型之间这种「展示态 ↔ 存储态」的自由转换，就是 Binding 手工构造的典型战场。

`@ViewBuilder trailing:` 参数是另一个进阶点——让调用方用 SwiftUI 语法块传入「日期下方的附加内容」（星座、相伴天数），这个写法叫 **ViewBuilder 尾随闭包**，你可以先跳过，用到时再学。

四、窗口的呈现：AppKit 里嵌 SwiftUI

设置面板是 SwiftUI 写的，但应用主体是 AppKit。两者怎么合体？答案在第 02 篇预告过的 **NSHostingController**——把 SwiftUI 视图装进 AppKit 窗口：

```
// AppDelegate 里的呈现逻辑（节选）
let controller = NSHostingController(rootView: SettingsView(model: settings))
settingsPanel.contentViewController = controller
settingsPanel?.makeKeyAndOrderFront(nil)
```

`NSHostingController` 是 SwiftUI ↔ AppKit 的官方桥：给它一个 SwiftUI 视图，它就能像普通 AppKit 视图控制器一样塞进 `NSWindow`。所以本项目的真实技术栈是：**AppKit 为骨（窗口、面板、事件），SwiftUI 为皮（设置、统计界面）**。这样的混搭在 mac 开发里极其常见——老代码增量迁移到 SwiftUI 就靠这座桥。

五、本章知识地图

知识点	位置	一句话记住它
UserDefaults	SettingsModel 全文	轻量设置的系统字典，`object(forKey:) as? T ?? 默认值` 是安全读法
属性观察器 didSet	每个 @Published 属性	「每次赋值后顺手做点什么」
@Published	同上	「这值变了，SwiftUI 你刷新一下」
onChange 回调	`var onChange: (() -> Void)?`	「设置变了，AppDelegate 你来应用」
手工 Binding	nameBinding / birthdayBinding	「读写去哪我说了算」的双向通道
GroupBox / Stepper / Toggle / Picker	SettingsView body	mac 设置界面的四大件
NSHostingController	AppDelegate 呈现处	SwiftUI 视图住进 AppKit 窗口的门

练习

1. 在 SettingsModel 里加一个 `@Published var themeColorName: String`（默认 "blue"），写进 UserDefaults，并在设置面板加一个 `Picker` 选颜色。体会「模型加一行、界面加一个控件、AppDelegate 接一次 onChange」的三步走。
2. 把 `optionalDatePicker` 里开关打开时的默认值从「今天」改成「2000 年 1 月 1 日」，想想为什么这样有时更好（提示：生日场景）。

3. 思考题：如果删掉 `didSet` 里的 `notify()`，只在 `@Published` 上保留依赖，界面还会刷新吗？引擎时长还会即时生效吗？分别会发生什么？动手验证。
4. 进阶：研究 `Zodiac.of(_:)`（星座计算，Zodiac.swift），它如何把日期映射到星座区间？尝试给它写一个单元测试。

11 · 统计系统：JSON 存储与快照算法

FocusPet 零基础开发教程 · 第 11 篇
对应源码：Sources/FocusPet/StatsStore.swift（112 行）、Sources/FocusPet/StatsView.swift（95 行）

番茄钟跑了一周，用户自然会问：「我这周专注了多久？连续几天了？」统计系统要回答这类问题。它的架构分两层：

```
数据层：StatsStore —— 存取原始记录（JSON 文件）
↓ snapshot() 加工
展示层：StatsView —— 拿加工好的快照画界面
```

两层的分界线是一个关键设计：**StatsSnapshot（快照）**。视图从不直接接触原始记录数组，只消费一个算好的、扁平的统计结果。为什么这样设计？看完本章你就明白。

一、数据层：一个 JSON 文件就是数据库

1.1 数据模型：Codable 的魔力

一条专注记录长这样：

```
/// 一条会话记录
struct SessionRecord: Codable {
var id: UUID = UUID()
var kind: String // "focus" / "shortRest" / "longRest"
var startedAt: Date
var endedAt: Date
var plannedSeconds: TimeInterval
var completed: Bool // 自然走完 = true；提前结束 = false

var actualMinutes: Int {
max(0, Int(endedAt.timeIntervalSince(startedAt) / 60))
}
}
```

注意第一个词：Codable。它是 Swift 里一个协议组合（Encodable + Decodable），效果是——**编译器自动为你生成「对象 ↔ 数据」的双向转换**。你不需要写任何序列化代码，只要让结构体的每个成员本身可编码（String、Date、UUID、Double、数组……全都可以），整个结构体就能一键存取。

这就是 Swift
式的「约定优于配置」：换别的语言，同样的功能往往要写一堆映射代码或引入一个第三方库。

actualMinutes 是个**计算属性**（只有 getter）：不存数据，每次访问时现场算「从结束到开始过了几分钟」。因为「实际分钟」永远可以由起止时间推导出来，存下来反而会不一致。

1.2 存取三行核心代码

```
/// 简单的 JSON 文件存储：~/Library/Application Support/FocusPet/sessions.json
final class StatsStore {
static let shared = StatsStore() // 单例：全应用一个仓库

private let url: URL // 数据文件地址
private(set) var records: [SessionRecord] = []

private init() { ... }

private func load() {
guard let data = try? Data(contentsOf: url) else { return }
records = (try? JSONDecoder().decode([SessionRecord].self, from: data)) ?? []
}

private func save() {
try? JSONEncoder().encode(records).write(to: url, options: .atomic)
}
}
```

读和写各一行核心调用，但每一行都值得咀嚼：

代码	拆解
`try? Data(contentsOf: url)`	读文件。`try?` 把「可能失败」变成「失败就返回 nil」——文件不存在？返回 nil，跳过整段。首次启动没有历史数据，这不是错误，是常态
`JSONDecoder().decode([SessionRecord].self, ...)`	把 JSON 字节翻译回对象数组。`[SessionRecord].self` 是「一个 SessionRecord 数组的类型本身」
`?? []`	解码失败（文件损坏、版本不兼容）也别崩溃，给个空数组当什么都没发生过
`.write(to: url, options: .atomic)`	**原子写入**：先写到一个临时文件，写完后一次性改名为正式文件。中途断电/崩溃也不会留下半个文件

全链路没有一处会抛异常到用户脸上——**读不到就当空的，写失败就当没写，文件坏了就从头再来**。统计数据丢了不致命，崩溃才致命。这是「优雅降级」思想的又一次实践（第 09 章通知授权那节见过）。

1.3 为什么不用 UserDefaults / CoreData / SQLite

你可能会问：第 10 章的 UserDefaults 不也是存数据吗？选型逻辑是这样的：

方案	适合	本项目为什么没选
UserDefaults	十几个扁平设置项	记录会无限增长，放偏好文件里会拖慢读取，也违背「偏好 ≠ 数据」的惯例
CoreData / SQLite	海量数据、复杂查询、多表关联	杀鸡用牛刀——我们的「查询」就是遍历数组算个和，1000 条记录的眼睛都眨一下
JSON 文件	结构简单的追加型记录，数据量中小	读写各一行，肉眼可打开检查，备份就是一个文件

工程选型的第一原则不是「哪个最先进」，而是「哪个刚刚好」。

1.4 单例的合理场景

```
static let shared = StatsStore()
private init() { ... }
```

`static let shared` + `private init` 是单例的标准写法：全局唯一实例，外部无法另行创建。第 05 篇提过单例容易藏耦合，为什么这里敢用？因为仓库**天生就该唯一**——两份各自为政的 records 数组意味着数据分裂，那才是真正的灾难。判断单例是否合理的试金石：**这个事物的「唯一性」是需求本身，还是偷懒图方便？** StatsStore 属于前者。

二、加工层：snapshot() 把数组翻译成答案

StatsView 需要六个数字（今日次数、今日分钟、近 7 天分钟、累计分钟、连续天数、近 7 天分布）。`snapshot()` 的工作就是把原始记录数组**一次性算出这六个数字**，打包成快照：

```
struct StatsSnapshot {
var todayCount = 0
var todayMinutes = 0
var weekMinutes = 0
var totalMinutes = 0
var streakDays = 0
var last7Days: [Int] = Array(repeating: 0, count: 7) // [0]=6天前 … [6]=今天
}
```

2.1 核心循环：一次遍历，四个统计

```
for record in records where record.completed && record.kind == "focus" {
let minutes = record.actualMinutes
let day = cal.startOfDay(for: record.endedAt) // 该记录归属哪一天（按结束时刻）

if day == todayStart {
snap.todayCount += 1
snap.todayMinutes += minutes
}
if day >= weekAgo {
snap.weekMinutes += minutes
let offset = cal.dateComponents([.day], from: day, to: todayStart).day ?? -1
if (0...6).contains(offset) {
snap.last7Days[6 - offset] += minutes // 落进对应的天槽
}
}
snap.totalMinutes += minutes
}
```

三个值得学的点：

1. for ... where **条件过滤**：只统计「完成且是专注」的记录。中断的专注（completed == false）和休息都不计入——产品规则一行声明式表达。
2. Calendar.current.startOfDay：把任意时刻归一到「那天的 0 点」。日期比较必须先归一，否则「今天 23:59 结束」和「今天 00:01 开始」会被当作不同的时间点而无法参与相等比较。这是所有日历类编程的第一课：**别比较时刻，比较「天」**。
3. **offset → 槽位映射**：dateComponents([.day], from: day, to: todayStart) 算出「那天距今天几天」，然后 last7Days[6 - offset] 落位——6 天前偏移 6，放槽 0；今天偏移 0，放槽 6。数组索引和「天前数」方向相反，这个换算容易绕晕，画个表就清楚了：

槽位	0	1	2	3	4	5	6
含义	6 天前	5 天前	4 天前	3 天前	前天	昨天	今天
offset（距今天）	6	5	4	3	2	1	0
索引公式	6−6	6−5	…	…	6−2	6−1	6−0

2.2 连续天数算法：今天没专注，从昨天起算

「连续专注 N 天」最常见的纠结：**今天还没专注， streak 要不要清零?**
合理的体验是：不清零，今天只是还没开始，昨天的记录仍然算数。streakDays 的算法体现了这个体贴：

```
var streakDays: Int {
let cal = Calendar.current
// 第一步：收集所有「完成过专注」的日期，去重成集合
let daysWithFocus = Set(
records
.filter { $0.completed && $0.kind == "focus" }
.map { cal.startOfDay(for: $0.endedAt) }
)
var count = 0
var day = cal.startOfDay(for: Date())
// 今天还没专注 → 起点挪到昨天（今天中断不算断签）
if !daysWithFocus.contains(day),
let yesterday = cal.date(byAdding: .day, value: -1, to: day) {
day = yesterday
}
// 从今天（或昨天）开始往前一天一天数
while daysWithFocus.contains(day) {
count += 1
guard let prev = cal.date(byAdding: .day, value: -1, to: day) else { break }
day = prev
}
return count
}
```

三个技巧：

- `Set` **去重**：同一天专注 5 次，集合里只有 1 个元素。连续天数数的是「天数」不是「次数」。
- **起点修正**：今天不在集合里，起点先挪到昨天。`if` 后面跟 `,` 再接一个 `let` 是 Swift 的「可选绑定 + 条件复合」语法——两个条件都满足才进分支。
- **回溯循环**：从起点逐日往前查集合，断了就停。循环次数 = streak 天数，天然高效。

三、展示层：纯 SwiftUI 的统计页

StatsView 是项目里第二处 SwiftUI（第一处是设置面板），它展示了一种典型的函数式视图风格——**只读输入，无副作用**：

```
struct StatsView: View {
let stats: StatsSnapshot // 只读！没有 @ObservedObject，没有 @State
```

视图拿到的只是一个快照值。数据从哪来、什么时候刷新，视图一概不管——每次打开统计窗口，AppDelegate 现场调一次 `snapshot()`、现场构建一个 StatsView。窗口关闭，视图销毁。没有订阅、没有同步、没有状态管理，**生命周期最简单的界面反而最不容易出 bug**。

3.1 柱状图：SwiftUI 画图表的朴素之道

界面最下面的近 7 天柱状图没有引用任何图表库，纯基础控件拼装：

```
HStack(alignment: .bottom, spacing: 10) { // 底对齐 → 柱子底部在一条线上
ForEach(0..<stats.last7Days.count, id: \.self) { i in
let minutes = stats.last7Days[i]
VStack(spacing: 4) {
Text("\(minutes)") // 柱顶数字
RoundedRectangle(cornerRadius: 3) // 柱子本体
.fill(i == 6 ? Color.orange : Color.gray.opacity(0.45)) // 今天橙色高亮
.frame(width: 26,
height: max(4, CGFloat(minutes) / CGFloat(max(1, stats.last7Days.max() ?? 1)) * 90))
Text(Self.dayLabel(daysAgo: 6 - i)) // 柱底日期
}
}
}
```

柱子高度的归一化公式值得单独看一眼：

```
height = minutes / max(1, 近7天最大值) × 90
```

- 最高的柱子永远是 90 点（满分柱），其余按与最大值的比例缩放——不管数据是 5 分钟还是 500 分钟，图都能看。
- 外面再包一层 `max(4, ...)`：0 分钟的柱子也有 4 点高，不至于「消失」导致对不齐。
- `max(1, ...)` 防除零：如果一周全没专注，最大值是 0，直接除会崩。

SwiftUI 画图的哲学和 Canvas 手绘（第 05 篇的气泡）不同：**用声明式布局控件当积木**，HStack 管横排、VStack 管纵排、`alignment:`

`.bottom` 一摆，柱状图自己就立起来了。适合条形类简单图表；折线图、饼图再考虑图表库。

3.2 复用小视图：StatBox

```
private struct StatBox: View {
let title: String
let value: String
var body: some View {
VStack(spacing: 4) {
Text(value).font(.title2.bold())
Text(title).font(.caption).foregroundStyle(.secondary)
}
.frame(maxWidth: .infinity) // 在 HStack 里均分宽度
.padding(.vertical, 10)
.background(RoundedRectangle(cornerRadius: 10).fill(Color.gray.opacity(0.12)))
}
}
```

一个大数字 + 一个小标题的统计卡片，在界面里用了 5 次。抽成独立的小视图后，5 次使用变成 5 行调用——这就是「组件化」的最小实践。`private` 修饰表示它是 StatsView 的私有小帮手，外界不需要知道它存在。

四、为什么是「快照」而不是「视图自己算」

最后回到开头的架构问题，现在可以给出完整答案了：

方案	问题
StatsView 直接持有 records 数组自己算	视图里塞满日期计算和过滤逻辑，界面代码被业务逻辑污染；换界面（比如菜单栏显示今日分钟）要重抄一遍算法
snapshot() 一次算好	算法集中在 StatsStore（数据的家）；StatsView 只画数字；以后任何新界面消费同一个快照即可

这条原则叫「**单一数据真相源**」（Single Source of Truth）：原始记录只有一个家（StatsStore），所有派生数字都从它那里算，绝不另起炉灶存第二份。

练习

1. 在 Playground 里造 10 条假记录（跨 5 天，含一次中断），跑 `snapshot()`，手算一遍期望值再对答案。
2. 给 StatsSnapshot 加一个 `averageMinutesPerFocus`（累计分钟 ÷ 累计完成专注数），分母为 0 时返回 0。想好这个除法该怎么防崩。
3. 把柱状图的最高柱从 90 改成 60，再把今天的颜色从橙色改成粉色——体会「改视图不改逻辑」的清爽。

4. 思考题：如果记录增长到 10 万条，`snapshot()` 每次全量遍历会不会慢？（提示：算一下 10 万次简单比较需要多少毫秒，再想想什么时候才值得优化。）

12 · AI 素材生产管线：从一句话到 144 帧动画

FocusPet 零基础开发教程 · 第 12 篇
对应素材：`Sources/FocusPet/PetAssets/`（144 张 PNG 序列帧）

这一章有点特别：前面十一篇讲的都是写代码，这一篇讲**怎么造出代码要用的素材**。FocusPet 里猫咪和狗狗的全部动画不是美术软件画的，而是 AI 生成的。最终成果：

宠物	动画组	每组帧数	小计
猫咪	idle / walk / run / sleep / focus / celebrate / wave / sad / eat	8	72
狗狗	同上，文件名加 `dog_` 前缀	8	72
合计	18 组		**144 帧**

这一章把整套管线拆开：提示词怎么写 → 一次生成 8 帧 → 切片 → 去底 → 修洞。每一步都有真实踩过的坑。

一、总体管线

```
① 写提示词 → ② AI 生成一张 8 格网格图 → ③ 按统一网格切成 8 帧
↓ ↓
④ 白底转透明（去底）→ ⑤ 修透明洞 → ⑥ 按 前缀_序号.png 命名入库
```

为什么要「一张图 8 格」而不是直接生成 8 张单独的图？两个原因：

1. **一致性**。让 AI 分别画 8 张「同一只猫」，每张都会长不一样；而一张图里的 8 格，角色基本是同一个。动画帧之间角色形状必须稳定，否则播起来像「跳变」而不是「动」。
2. **效率**。生成一张大图比生成 8 次、每次还要祈祷角色不变，省时得多。

二、提示词的写法

提示词（prompt）是与 AI 绘图模型沟通的语言。写动画帧提示词的核心原则是：**把「美术要求」全部说死，把「变化」只留给动作本身**。

一套可直接套用的模板（本项目实际使用的写法，可按你的宠物随意改）：

```
A cute kawaii [橘色虎斑小猫 / 棕色小狗], [动作描述，如：idle animation,
gentle breathing with tail swaying], simple flat cartoon style, thick clean
outline, full body centered in each cell, consistent character design across
all cells, pure white background, no shadow, no watermark, no text,
4x2 grid layout with 8 equal cells, each cell shows one frame of the animation
in sequence
```

逐块拆解，每块都对应一个真实的需求：

提示词片段	防止的灾难
kawaii ... simple flat cartoon style	防止 AI 画成写实风（写实风序列帧有一点抖动就瘆人）
consistent character design across all cells	防止 8 格画出 8 只不同的猫
full body centered in each cell	防止角色出框——出框的帧切出来就是「缺腿猫」
pure white background, no shadow	为下一步「去底」创造条件：纯白底最容易程序化移除
no watermark, no text	防止画面角落出现乱码字母（AI 签名癖）
4x2 grid layout with 8 equal cells	让 8 帧排成规则网格，程序才能按统一坐标切

动作描述的措辞

每个动画组只改模板里的动作部分。写动作时的关键认知：**AI 不理解「动画」，它理解的是「8 个略有差别的静态姿势」**。所以动作描述要描述「这一组姿势整体在干什么」，而不是「从 A 到 B 的过程」。实际使用过的描述风格：

组	动作描述思路
idle	轻轻呼吸、尾巴缓慢摆动（变化极小，8 帧几乎像静图）
walk	走路循环，四腿交替迈步（猫：慢、慵懒；狗：快、蹦跳感）
run	快速奔跑，身体拉伸
sleep	趴着，Zzz 式起伏（可以偶尔带一个呼噜气泡，但注意别带文字）
focus	正襟危坐、表情认真（配合番茄钟专注状态）
celebrate	跳跃、双爪举起、开心
wave	举爪招手
sad	垂耳低头、沮丧（用于用户中途放弃专注）
eat	低头啃食盆、咀嚼

猫和狗的**走路节奏差异**是在提示词阶段就定下的：猫的 walk 描述里加 slow, lazy, relaxed pacing，狗的 walk 里加 bouncy, energetic, happy trotting。这也是后来代码里猫帧时长 0.6 秒、狗 0.26 秒的底气（第 06 篇讲过）。

参考图：系列一致性的大招

画完猫之后要画狗、或者要画同一只猫的第二个动作组，最怕的是「两只猫」。解法是给 AI 一张**参考图**（image reference）：上传已生成、效果满意的角色图，让新图「照着这只画」。本项目所有动画组的角色一致性就是这么维持的。写提示词时把参考图放在最前面，再补一句 same character as the reference image。

三、切片：统一裁剪框

AI 交出的是一张 4×2 网格大图。切成 8 帧的程序化思路：

```
整张图宽 W、高 H，则每格宽 W/4、高 H/2
第 n 帧（n 从 0 数到 7）的格子坐标：
列 = n% 4 （第几列）
行 = n / 4 （第几行，整除）
裁剪框 = (列 × W/4, 行 × H/2, W/4, H/2)
```

用 Pillow（Python 图像库）十几行就能写完。要点只有两个：

1. **统一裁剪框**：所有帧组用同一套相对坐标切，保证切出来的角色位置一致——位置不一致，播放时角色会在原地「抖」。
2. **切完人工抽查**：程序切完，务必把 8 帧按顺序快速翻看一遍（Mac 上选中 8 张图按空格预览，来回按方向键）。AI 偶尔会画出 7 格或 9 格、或某格里角色出框，这类问题只有肉眼能发现，发现就重新生成这一组，别试图修。

四、去底与修洞：两个大坑

4.1 坑一：去底不能简单「去白」

切片后是白底。转透明（俗称去底）的最朴素想法：把所有接近白色的像素变透明。这里埋着本项目最阴的坑——**有些猫是乳白色、奶白色的**。「接近白」的阈值稍微激进一点，就会把猫身上的浅色毛发一起删掉，切出来的 PNG 上赫然出现**透明的洞**。

4.2 坑二：「智能去水印」会制造洞

还有一种看起来更高级的坑。AI 生成的图有时带模型水印或角落的乱码签名，会引入一种叫 clean_watermark 的去水印工具/功能。它的原理是识别「重复出现的图案」并抹除——问题就在「重复」二字：**动画帧里角色的重复出现被它当成了水印**，一抹，角色身上又多了透明区域。

4.3 修法：BFS 边界洪水填充

正确的去底姿势不是「全图删白」，而是**洪水填充（Flood Fill）**：

> 只从**图像四边**出发找连通的白色区域删除，图像内部的白色（哪怕和边缘白只有一条细线连着也得靠「连通」来判断）一律不碰。

用 BFS（广度优先搜索，第 01 篇提过队列思想）实现：

```
1. 把四边上所有白色像素入队，标记为「待删」
2. 循环：出队一个像素 → 检查它的上下左右四邻居
- 邻居是白色且未标记 → 入队、标记
3. 队列空后，把所有「待删」像素置为透明
```

因为搜索**只从边缘出发**，宠物身体内部的乳白毛发哪怕颜色和白底一模一样，也隔着一层有颜色的轮廓线， flood fill 根本到不了它——洞的问题从原理上被根除。

如果某些帧还是不幸带了洞（比如白底和毛发之间真有缺口连通），修复思路也基于同一个原理的逆运用：从洞的边缘往里填色，而不是从全图删色。

4.4 工程顺序的建议

> 先生成 → 再切片 → 先检查有没有水印 → 有水印重生成（别用去水印工具）
> → 边缘洪水填充去白底 → 抽查 → 入库

「能重新生成就别修图」是 AI 素材时代的黄金法则：提示词改一改重新抽一张，往往比 PS 半小时快得多、效果也好得多。

五、命名与入库：和代码的契约

最终文件名严格遵守代码的约定（第 05 篇的 `AnimationSpec`）：

> 猫咪：idle_1.png ... idle_8.png （无前缀 = 默认猫）
> 狗狗：dog_idle_1.png ... dog_idle_8.png

这个「猫无前缀、狗加 `dog_`」的约定意味着：**代码里所有猫咪的动画组一行不用改**，狗的素材加前缀就共存了。给素材命名时脑子里要有代码——`prefix` 对应文件名前缀，动画组在代码里加一个 `static let` 就接入完成（第 06 篇练习 3 做过这个）。

素材目录 `PetAssets/` 直接放在 `Sources/FocusPet/` 下，随 SwiftPM 打包（第 13 篇详解），`PetAssets.zip` 留一份压缩备份，方便整体迁移（比如后来做 Windows 版时直接拷走）。

六、给非美术程序员的整体建议

阶段	建议
提示词	说死一切风格要求，只留动作一个变量；同角色必用参考图
生成	一次不满意就重抽，提示词是彩票，多买几张
切片	统一裁剪框 + 人工逐帧抽查，两者缺一不可
去底	只信边缘洪水填充，不信「全图删白」和「智能去水印」
命名	命名前先读代码的文件查找逻辑，命名即契约

这套管线跑通一次后，新增一个动画组（比如后来的 eat）的成本大约是：写一段动作描述 → 生成 2~3 张挑一张 → 切片抽查 → 去底 → 放进目录 → 代码里加一个 `AnimationSpec`。**半小时内，宠物就学会了一个新动作。**

练习

1. 用本章的提示词模板，给你自己的桌面宠物生成一组 idle 动画（任何你顺手的 AI 绘图工具），按 4×2 网格要求检查输出。
2. 用 Pillow 写一个切片脚本：输入一张 4×2 网格大图，输出 8 张 `test_1.png` ~ `test_8.png`。
3. 实现边缘洪水填充去白底，并故意找一张有浅色主体（白云、白猫）的图，对比「全图删白」和「洪水填充」的结果差异。
4. 思考题：如果 AI 只愿意输出 3×3 九宫格，切片公式要怎么改？（提示：`%` 和 `/` 的除数变了。）

13 · 构建、打包与发布：从 swift run 到 /Applications

FocusPet 零基础开发教程 · 第 13 篇
对应源码：Package.swift（19 行）、package-app.sh（26 行）、Sources/FocusPet/Info.plist

到目前为止你写的代码都能用 swift run 跑起来。但 swift run 跑的是**裸可执行文件**——第 09 章说过，它连系统通知都用不了。要成为一个真正的 mac 应用，必须经过打包。本章把「编译 → 打包 → 部署 → 验证」整条链路走完。

一、先厘清三个名词

初学者最容易混淆的三个概念：

名词	是什么	FocusPet 里的对应物
SwiftPM 包（Package）	源码 + 构建清单的组织单位	整个 FocusPet 目录，Package.swift 是清单
可执行文件（Binary）	编译产物，一个 Mach-O 程序	.build/release/FocusPet
.app 包（Bundle）	目录伪装成的「应用」，系统认可的发布形态	FocusPet.app/（内含可执行文件 + 资源 + 元数据）

关键认知：.app **不是编译出来的，是「组装」出来的**。编译只产出一个二进制文件；把它放进标准目录结构、配好元数据（Info.plist）、带上资源，才成为一个应用。

二、Package.swift：构建清单

整个项目的构建配置只有 19 行：

```
// swift-tools-version: 5.9
import PackageDescription

let package = Package(
name: "FocusPet",
platforms: [
.macOS(.v14)
],
targets: [
.executableTarget(
name: "FocusPet",
path: "Sources/FocusPet",
exclude: ["Info.plist"],
resources: [
.copy("PetAssets")
]
)
]
)
```

逐项拆解：

配置	含义
`swift-tools-version: 5.9`	本包使用的 Swift 工具链最低版本，写在文件最顶上
`name`	包名，也是默认产物名（FocusPet 可执行文件）
`platforms: [.macOS(.v14)]`	最低支持 macOS 14（Sonoma）。API 用到 14 的新特性时这里必须对齐
`.executableTarget`	可执行目标（对应还有 `.target` 库目标）。一个包可以有多个目标
`path: "Sources/FocusPet"`	源码目录（默认就是 `Sources/<目标名>`，写出来更明确）
`exclude: ["Info.plist"]`	Info.plist 不参与编译，但要放在目录里供打包脚本取用——告诉构建系统「忽略它」
`resources: [.copy("PetAssets")]`	**本章重点**，下面单讲

resources 与 Bundle.module

```
resources: [
.copy("PetAssets")
]
```

这行告诉 SwiftPM：构建时把 `PetAssets` 目录原样拷贝进资源包。SwiftPM 会为每个带资源的 target 生成一个**配套的资源 bundle**（名字形如 `FocusPet_FocusPet.bundle`），构建产物里你会同时看到：

```
.build/debug/
├── FocusPet ← 可执行文件
└── FocusPet_FocusPet.bundle/ ← 资源包（里面装着 PetAssets/144 张 PNG）
```

代码里通过 Bundle.module 访问它（第 05 篇出现过）：

```
Bundle.module.url(forResource: "\(spec.prefix)_\(index)",
withExtension: "png",
subdirectory: "PetAssets")
```

.copy vs .process：前者原样拷贝（我们的 PNG 不需要任何处理），后者是 SwiftPM 的「智能处理」模式（会按平台优化图片等）。素材类资源用 .copy 最省事、行为最可预测。

三、Info.plist：应用的身份证

Info.plist 是一个 XML 格式的元数据文件，mac 系统在启动应用时要读它来回答一堆问题：这个包的可执行文件叫什么？bundle ID 是什么？图标是哪个？要不要显示在 Dock？

FocusPet 的完整 Info.plist 只有 30 行，关键键值：

键	值	管什么
CFBundleIdentifier	local.focuspet.app	全球唯一 ID，系统用它区分应用。通知授权、单实例保护都认它
CFBundleExecutable	FocusPet	Contents/MacOS/ 下哪个文件是要运行的程序
CFBundleShortVersionString	0.6	用户看到的版本号
LSUIElement	true	**菜单栏应用开关**：不显示 Dock 图标、不进 Cmd+Tab。第 09 章那个「点击菜单要先 activate」的坑就是它带来的
CFBundleIconFile	AppIcon	应用图标（Contents/Resources/AppIcon.icns）
LSMinimumSystemVersion	14.0	最低系统版本，与 Package.swift 的 platforms 呼应

注意它在 Package.swift 里被 exclude 了——因为它不是编译输入，而是**打包阶段的装配原料**。这是 SwiftPM 项目的常见手法：plist 放源码目录里跟着版本管理，打包时拷进 .app。

四、package-app.sh：把零件组装成 .app

swift build 只给你零件（二进制 + 资源 bundle + plist），package-app.sh 负责组装。逐段读：

```
#!/bin/bash
# 把 SwiftPM 产物包装成可运行的 .app（UserNotifications、自启等功能需要正式 bundle）
# 用法: ./package-app.sh [debug|release] （默认 debug）
set -e
cd "$(dirname "$0")"
```

- `set -e`：任何命令失败立即退出脚本，不让错误静默累积。
- `cd "$(dirname "$0")"`：无论你在哪个目录调用这个脚本，都先切到脚本所在目录（项目根）。`$0` 是脚本自身的路径。

```
CONFIG="${1:-debug}"
APP=.build/$CONFIG/FocusPet.app
rm -rf "$APP"
mkdir -p "$APP/Contents/MacOS" "$APP/Contents/Resources"
```

- `"${1:-debug}"`：第一个命令行参数，缺省值 debug。`./package-app.sh release` 就出 release 版。
- `rm -rf` + `mkdir -p`：每次都从空目录组装，杜绝「上次打包的残留混进来」这种幽灵 bug。

```
cp .build/$CONFIG/FocusPet "$APP/Contents/MacOS/FocusPet"

# SwiftPM 资源包（帧动画素材）：Bundle.module 在 Bundle.main.resourceURL
# （即 Contents/Resources/）下查找
for b in .build/$CONFIG/*.bundle; do
[ -e "$b" ] && cp -R "$b" "$APP/Contents/Resources/"
done

cp Sources/FocusPet/Info.plist "$APP/Contents/Info.plist"
```

这是 .app 目录结构的填充。标准 .app 布局：

```
FocusPet.app/
└── Contents/
├── Info.plist ← 应用的身份证
├── MacOS/
│ └── FocusPet ← 可执行文件
└── Resources/
├── AppIcon.icns ← 图标
└── FocusPet_FocusPet.bundle/ ← SwiftPM 资源包（素材在里面）
```

注释里那句是血泪知识：`Bundle.module` **在运行时是从** `Bundle.main.resourceURL` **往下找的**——`Bundle.main` 是 .app 本身，它的 resourceURL 就是 `Contents/Resources/`。所以资源 bundle 必须落在 Resources/ 下，放错位置，144 张素材一张都找不到（PetView 会打出一排 `缺少素材`）。

```
cp "$(dirname "$0")/../FocusPetAssets/AppIcon.icns" \
"$APP/Contents/Resources/AppIcon.icns" 2>/dev/null || true

# ad-hoc 签名（通知投递对签名有要求）；失败不阻断，直接运行通常也可工作
codesign --force --sign - "$APP/Contents/MacOS/FocusPet" 2>/dev/null || true
```

两处 `|| true` 是同一个哲学：**非关键步骤失败不阻断打包**。图标文件不在？先出个没图标的包。临时签名失败？多数功能照样能跑。对单人小项目，「包先出来」比「每一步都完美」重要。

codesign --sign - 是什么

mac 的可执行文件需要「签名」才能享受全部系统服务（通知就是其一）。`--sign -` 是 **ad-hoc 签名**——「我自己给自己签，没有证书」。正式的开发者证书（Apple Developer Program，年费 99 美元）是用来发布到 App Store 或让他人无障碍运行的；自己机器上用 ad-hoc 足够。不签名或签名坏了的典型症状就是第 09 章那个：通知不工作，或系统弹「无法验证开发者」。

五、部署到 /Applications 并验证

打包出 `.build/release/FocusPet.app` 后，部署标准位置是 `/Applications`：

```
# 关掉运行中的旧版本，替换，重新打开
killall FocusPet 2>/dev/null || true
rm -rf /Applications/FocusPet.app
cp -R .build/release/FocusPet.app /Applications/
open /Applications/FocusPet.app
```

验证小技巧

1. MD5 验证「我拷过去的到底是不是刚编译的」

更新部署后如果行为没变，先别怀疑代码——可能是拷贝没生效或跑了旧进程：

```
md5 .build/release/FocusPet.app/Contents/MacOS/FocusPet
md5 /Applications/FocusPet.app/Contents/MacOS/FocusPet
```

两个哈希一致 = 文件确实是新的。不一致 = 部署环节出了问题。

2. killall Dock 刷图标缓存

更换 AppIcon 后，Finder 和 Dock 可能顽固地显示旧图标——那是系统的图标缓存没刷新：

```
killall Dock # Dock 重启并重建缓存
killall Finder # Finder 同理
```

若还不行，更彻底的方案是删 `~/Library/Caches/com.apple.iconservices.store` 后重启，但一般情况下重启 Dock 就够了。

3. Release vs Debug 的行为差

`swift build` 默认 debug：不做优化、带调试符号，**运行明显更慢更费电**。正式使用务必 `swift build -c release` 后再打包。省电版教程（第 14 篇）会再次强调这一点。

六、发布形态总结

```
源码 FocusPet/
 │
 │ swift build -c release
▼
零件：.build/release/FocusPet（二进制）
.build/release/FocusPet_FocusPet.bundle（素材包）
 │
 │ ./package-app.sh release
▼
成品：.build/release/FocusPet.app（标准 bundle）
 │
 │ cp -R → /Applications
▼
运行中的菜单栏宠物
```

想分发给朋友？把 .app 压进 dmg 或 zip 即可；对方下载后可能需要在右键菜单里选「打开」以绕过 Gatekeeper（未付费公证的应用的必经一步）——这就是个人开发者发布 mac 应用的完整最小闭环。

练习

1. 故意把 `Info.plist` 的 `LSUIElement` 改成 `false`，重新打包运行，观察 Dock 的变化；再改回来。
2. 打包后把 `Contents/Resources/` 下的资源 bundle 改名，运行应用看 PetView 打印什么——亲眼看一次 `Bundle.module` 找不到资源的报错。
3. 用 `md5` 对比 debug 和 release 两个二进制，再对比各自的文件大小（`ls -lh`），体会 debug 携带的调试符号有多占地方。
4. 进阶：给 package-app.sh 加一行，打包后自动 `cp -R` 到 /Applications（记得先 killall）。体会「脚本化部署」省掉的机械劳动。

14 · 加餐：投喂饥饿系统与省电优化

FocusPet 零基础开发教程 · 第 14 篇（完结篇）
对应源码：Sources/FocusPet/AppDelegate.swift（饱腹状态部分）、Sources/FocusPet/PetController.swift（投喂/讨食部分）、Sources/FocusPet/PetView.swift（省电重绘部分）

正篇到此全部讲完。最后这章加两道「餐后甜点」：一是项目收官前上线的**投喂/饥饿系统**——它是前面几乎所有知识点的综合演习；二是开发过程中被反复追问的**省电优化**——桌面宠物 7×24 小时驻留，功耗是真实的产品质量。两道甜点吃完，整套教程的知识就闭环了。

一、投喂与饥饿：需求的完整拆解

收官前加的需求，原话是：「加一个点击投喂吃的效果，然后每两个小时就自己出来找吃的，或者提醒主人给他喂吃的。」把这句口语翻译成工程规格：

口语	工程规格
「点击投喂吃的效果」	单击宠物 → 播 eat 动画（有限循环）→ 吃完回当前状态
「每两个小时就……」	距上次投喂超过 2 小时 → 进入饥饿状态
「自己出来找吃的」	空闲时饿了 → sad 表情 + 饥饿气泡 + 随机移动讨食
「提醒主人给他喂吃的」	专注中饿了 → 发系统通知（但不能轰炸）

注意最后一条的产品微妙之处：**专注中宠物不该跳出来打扰你**（这正是番茄钟产品的价值观），所以它「努力不打扰你」，只发一条克制的系统通知。这个「分场景不同行为」的设计，直接决定了代码结构。

二、状态归属：谁管「饿不饿」

第一个架构决策：**饥饿状态放谁身上？**

答案藏在两个类的职责划分里。PetController 管「宠物的表现」（动画、移动、气泡），AppDelegate 管「全局事实」（时间、通知、设置）。「距上次投喂 2 小时」是一个时间事实——所以**计时与判定在 AppDelegate，状态表现的分发在 PetController**。两边靠一个属性和两个回调连接：

```
// AppDelegate：时间事实的持有者与判定者
private static let lastFedKey = "lastFedAt"
private static let hungerThreshold: TimeInterval = 2 * 3600 // 2 小时（秒）

/// 每 30 秒检查一次宠物是否饿了
private func startHungerCheck() {
hungerCheckTimer?.invalidate()
hungerCheckTimer = Timer.scheduledTimer(withTimeInterval: 30, repeats: true) { [weak self] _ in
self?.updateHungerState()
}
updateHungerState()
}

private func updateHungerState() {
pet?.isHungry = Date().timeIntervalSince(lastFedAt) > Self.hungerThreshold
}
```

```
// PetController：拿到状态变化后决定怎么表现
var isHungry = false {
didSet {
guard isHungry != oldValue else { return } // 状态没变就不动
if isHungry {
scheduleBeg() // 饿了：安排讨食
} else {
hungerWorkItem?.cancel() // 喂饱了：取消讨食
petView.updateHunger(nil) // 撤掉气泡
}
}
}
```

这段代码里每一行都是前面学过的知识在值班：

- `static let` 常量：阈值这类「魔法数字」必须命名（第 01 篇）。
- `Timer` 常驻 + 轮询：不需要精确到秒的判定，30 秒查一次绰绰有余，比「精确闹钟」简单一百倍。
- `Date().timeIntervalSince(lastFedAt)`：两个时刻相减得秒数（第 11 篇的日期计算同款）。
- `didSet` 属性观察器：值一变自动触发表现逻辑（第 10 篇的设置项同款）。
- `guard isHungry != oldValue`：只在变化时行动（第 08 篇 setPhase 同款礼仪）。

而 `lastFedAt` 本身的读写封装是另一个 Swift 小知识：

```
private var lastFedAt: Date {
get {
let ts = UserDefaults.standard.object(forKey: Self.lastFedKey) as? TimeInterval
return ts.map(Date.init(timeIntervalSince1970:)) ?? Date()
}
set {
UserDefaults.standard.set(newValue.timeIntervalSince1970, forKey: Self.lastFedKey)
}
}
```

计算属性 + UserDefaults 的组合：对外表现得像普通属性（`lastFedAt = Date()`），对内每次读写都落盘。进程重启后投喂记录不丢。`?? Date()` 的默认值很妙：从没喂过 → 视为「刚刚喂过」（现在才出生，当然不饿）——首次启动不会让宠物一睁眼就讨饭。

三、投喂：打断与恢复的艺术

`feed()` 是 PetController 里最忙的方法，因为它要处理「宠物正在做任何事时被喂」的所有组合：

```
/// 投喂：打断当前动作，播放吃动画，吃完回状态
func feed() {
guard !isPlaying(.eat) else { return } // 正在吃就别重复喂
hungerWorkItem?.cancel() // 取消排队中的讨食
hungerBubbleWorkItem?.cancel() // 撤讨食气泡
petView.updateHunger(nil)
// 打断移动
if roamAnim != nil {
roamAnim?.invalidate()
roamAnim = nil
roamFrom = nil
roamTo = nil
onArriveMove = nil
}
petView.bounce()
petView.showHearts(count: 3)
play(.eat) { [weak self] in
guard let self else { return }
self.onFed?() // 告诉 AppDelegate：重置饱腹计时
if self.isSessionActive() {
self.play(self.inRestSession ? .sleep : .focus) // 吃完回会话状态
} else if self.isPlaying(.eat) {
self.play(.idle) // 吃完回待机
}
}
}
```

拆解它的动作序列：

1. **防御**：正在吃就忽略（防抖）；取消所有讨食相关任务。
2. **打断**：如果在移动，拆掉移动定时器并清掉所有移动状态（包括 `onArriveMove`——庆典移动被打断时不能到了「目的地」还触发改庆祝）。
3. **仪式感**：弹跳一下 + 飘 3 颗爱心——食物是幸福的（复用第 07 章的爱心粒子，白捡的仪式感）。
4. **吃完收尾**（completion 回调）：先通知 AppDelegate 重置计时，再根据**当前场景**决定回哪个动画——会话中回 focus/sleep，空闲回 idle。

最后那个 completion 里有个一眼看不太懂、实则精妙的守卫：

```
} else if self.isPlaying(.eat) {
self.play(.idle)
}
```

为什么是 `isPlaying(.eat)`？想象这个时序：宠物还在吃，用户开始了专注 → `onSessionStart` 把它切到 focus 动画 → eat 播完的 completion 触发。如果没有这层检查，它会**把专注中的宠物拉回 idle**——bug。有了这层检查：「只有当我还停留在 eat 动画（没人打断我）时才回 idle」，优雅地免除了时序竞争。

「吃完回哪个状态」不由 feed 自己记住「吃之前是什么」，而是**吃完那一刻重新查询全局状态**（`isSessionActive()` / `inRestSession`）。这是与第 08 章热键同一招：不问历史，只查现状。状态机的状态是唯一的真相源，记住它比自己记一份「之前的动作」可靠得多。

四、讨食：分场景的两种剧本

饿了的宠物不会一直讨，而是每隔随机一段时间「提醒一次」，由 `scheduleBeg` 排班：

```
/// 排下一次讨食
private func scheduleBeg(delay: TimeInterval? = nil) {
hungerWorkItem?.cancel()
let wait = delay ?? Double.random(in: 60...180) // 每 1~3 分钟提醒一次
let item = DispatchWorkItem { [weak self] in
self?.begForFood()
}
DispatchQueue.main.asyncAfter(deadline: .now() + wait, execute: item)
hungerWorkItem = item
}
```

讨食剧本在 `begForFood()` 里分场景：

```
private func begForFood() {
guard isHungry, !isPlaying(.eat) else { return }
scheduleBeg() // 先排下一次（无论这次讨没讨成）
if isSessionActive() {
onHungryDuringSession?() // 专注中：只发通知，不闹
return
}
// 空闲：委屈表情 + 气泡 + 四处找吃的
play(.sad) { ... 回 idle ... }
petView.updateHunger("我饿了，点击喂我 ")
// 8 秒后自动撤气泡
...
performRoam() // 饿得四处乱走
}
```

两个设计细节：

1. **先排下一次再讨这次**。如果讨食中途被打断（比如用户正好点了投喂），下一次提醒照样在队列里，讨食节拍不断。
2. **专注中的通知有独立节流**（AppDelegate 侧）：

```
/// 专注中饿了：发系统通知提醒（30 分钟最多一条，避免轰炸）
private func notifyHungryThrottled() {
if let last = lastHungerNotifyAt, Date().timeIntervalSince(last) < 1800 { return }
lastHungerNotifyAt = Date()
notify(title: "\(settings.currentPetName)饿了 ",
body: "它正在努力不打扰你，结束后记得点击它喂点吃的～")
}
```

注意节流**放在 AppDelegate 而不是 PetController**——因为「通知」是 AppDelegate 的职责。PetController
只知道「饿了想提醒」，至于怎么提醒、提醒多少次，不归它管。分层又一次挡住了复杂度。

五、省电优化：三层防线

桌面宠物是常驻进程，用户开着它工作一整天。动画没做好，CPU 占用能到 5%~10%，风扇嗡嗡叫——产品立刻被判死刑。FocusPet 的省电是**三道层层递进的防线**：

防线一：没变化就不重绘（PetView 内）

第 05 篇讲过的核心机制，这里值得从省电角度再强调一次：

```
// 省电：画面内容没变就不重绘（静止帧 / 未到下一帧时刻）
if frameIndex != drawnFrameIndex {
drawnFrameIndex = frameIndex
needsDisplay = true
}
```

`needsDisplay = true` 是整个绘制系统的「油门」——不设它，`draw(_:)` 一次都不会执行。序列帧动画的帧率是 0.28 秒一帧，意味着**每秒钟最多重绘 3~4 次**，其余时间即使 DisplayLink 在跑，视图纹丝不动。对比 naive 实现（每帧都重绘 = 120Hz 满血刷），CPU 占用差一个数量级。

防线二：没活干就不派活（DisplayLink 层）

DisplayLink
在独立线程以屏幕刷新率（120Hz）回调——回调本身也有成本。所以回调第一关：

```
guard view.hasPendingWork(at: now) else { return kCVReturnSuccess }
```

```
/// 指定时刻是否有画面要变：到下一帧时刻 / 弹跳中 / 爱心飘浮中
private func hasPendingWork(at now: CFTimeInterval) -> Bool {
if !frames.isEmpty && !finished {
return now >= frameStartTime + frameDurations[frameIndex]
}
return bounceT0 != nil || !hearts.isEmpty
}
```

没到下一帧的时刻、没有弹跳、没有爱心 → 连 `DispatchQueue.main.async` 都不派发。120Hz 的回调大多在独立线程里直接返回，主线程零打扰。

防线三：彻底歇业（窗口隐藏时）

```
/// 窗口隐藏时调用：彻底暂停渲染省电
func pauseRendering() {
stopDisplayLink()
}

/// 窗口重新显示时调用：有未完成动画则恢复刷新
func resumeRendering() {
if !finished || bounceT0 != nil || !hearts.isEmpty {
startDisplayLink()
}
}
```

菜单里有「显示/隐藏宠物」开关。隐藏时不是「画一张静止图」，而是**直接把 DisplayLink 停掉**——线程回调归零，CPU 占用趋近于零。重新显示时如果动画早已播完（idle 无限循环除外，它在播），甚至不需要重启 DisplayLink。

防线之外：两个习惯

- **Release 构建**。Debug 版不做编译优化，同一份代码 CPU 占用可能高 2~3 倍。自己日常用也必须 `swift build -c release`（第 13 篇）。
- **定时器的节制**。项目里所有 Timer（乱窜调度、移动动画、饥饿检查）都是「要么不建、建了就按需触发」，没有一个是无意义空转的。

用活动监视器验证效果： release 版 FocusPet 日常 CPU 占用通常在 0%~1% 之间浮动——对一颗常驻桌面的小宠物来说，这就是合格的数字。

六、全书知识总复盘

到这里，15 篇教程全部结束。回看这个 4000 多行的项目，它用到的全部核心知识其实就一张表：

知识域	关键词	所在篇
Swift 语言	可选值、guard、闭包、属性观察器、枚举、协议	01
应用骨架	AppKit/SwiftUI 分工、生命周期、 accessory 应用	02、09

知识域	关键词	所在篇
架构	类分工、回调解耦、状态机、单一真相源	03、08
窗口与绘制	NSPanel、透明窗口、needsDisplay、图层残影	04、05
动画	序列帧、CVDisplayLink、缓动、镜像转身	05、06
交互	performDrag、点击/双击判定、DispatchWorkItem	07
系统服务	状态栏、Carbon 热键、UserNotifications	09
数据	UserDefaults、Codable、JSON 原子写、快照	10、11
工程	SwiftPM 资源、plist、打包脚本、签名、部署	13
产品	AI 素材管线、投喂系统、省电三层防线	12、14

最重要的是那几条超越本项目的通用心法：

1. **状态机思维**——任何有「状态」的功能，先画状态图再写代码。
2. **回调解耦**——零件报事实，装配做决定。
3. **只在变化时行动**——不重绘、不通知、不算账。
4. **优雅降级**——读不到就当空的，没授权就静默，缺文件不崩溃。
5. **用真实代码当教材**——这套教程里每一行代码都能在项目里找到出处。

现在，轮到你从零开始，写出属于自己的那一个了。

练习

1. 把饥饿阈值从 2 小时改成 30 秒，`swift run` 观察讨食循环，然后改回来。（30 秒能让你在一杯咖啡的时间里看完整个饥饿状态机的一生。）
2. 给讨食气泡加一个新文案池（随机从三句「我饿了」的俏皮话里选），体会表现层的自由度。
3. 在活动监视器里对比 debug 版和 release 版的 CPU 占用，量化「防线之外」的第一个习惯。
4. 毕业设计：不参考任何现有代码，只凭状态图和本表，独立实现一个「宠物困倦系统」（深夜自动犯困、点击可以唤醒）——它几乎是饥饿系统的镜像，但「深夜」的判定要用 Calendar。

FocusPet 零基础开发教程 · 总索引

一套面向零基础读者的 macOS 桌面应用开发教程。
以真实项目 FocusPet（一只陪你专注的桌面宠物番茄钟，纯 Swift + AppKit/SwiftUI，约 4000 行）为教材，每一篇讲解都对应项目中**真实存在、可以直接打开对照**的源码文件。

教程假定你：会用 Mac，没写过 Swift，没开发过 mac 应用。

阅读路线

第一阶段：地基（第 00~03 篇）

建立整体认知：要做什么、语言怎么写、mac 应用由什么构成、类之间怎么分工。

篇	标题	你将学会	对应源码
00	开篇：你即将构建什么	项目全貌、功能清单、最终效果	全部
01	Swift 语法零基础速成	变量/可选值/guard/闭包/枚举——够用的最小语法集	全项目通用
02	macOS 应用解剖	AppKit vs SwiftUI、应用生命周期、事件循环	AppDelegate.swift、main.swift
03	架构总览：类的分工地图	每个类管什么、为什么这样分、回调解耦思想	全部类的鸟瞰

第二阶段：让宠物出现并动起来（第 04~06 篇）

项目的视觉核心：透明悬浮窗、序列帧动画、自主行为。

篇	标题	你将学会	对应源码
04	透明悬浮窗：宠物的舞台	NSPanel、无边框透明窗口、置顶与全桌面漫游	PetController.swift（窗口配置）
05	动画引擎：序列帧与省电重绘	CVDisplayLink、帧缓存、needsDisplay、图层残影之坑	PetView.swift（动画部分）
06	行为调度：让宠物活起来	随机行为调度、缓动插值、镜面转身、猫狗差异化	PetController.swift（行为部分）

第三阶段：用户与系统（第 07~09 篇）

交互、番茄钟内核、以及菜单栏应用的三件套。

篇	标题	你将学会	对应源码
07	交互系统：点击拖拽与双击	performDrag 零延迟拖拽、单击/双击的判定战争	PetView.swift（交互部分）
08	番茄钟引擎：状态机思维	状态机设计、回调通信、Timer 闸控防漂移	FocusEngine.swift（全 141 行）
09	状态栏、热键与通知	NSStatusItem、Carbon 全局热键、UserNotifications	StatusBarController.swift、HotkeyManager.swift

第四阶段：完整产品（第 10~13 篇）

设置、统计、素材生产、打包发布——从产品到交付的全部闭环。

篇	标题	你将学会	对应源码
10	设置面板与持久化	UserDefaults、@Published、SwiftUI 手工 Binding	SettingsModel.swift、SettingsView.swift
11	统计系统：JSON 存储与快照算法	Codable、原子写入、连续天数算法、SwiftUI 图表	StatsStore.swift、StatsView.swift
12	AI 素材生产管线	提示词写法、网格切片、去底修洞的坑	PetAssets/（144 帧素材）
13	构建、打包与发布	SwiftPM 资源、Info.plist、组装 .app、签名与部署	Package.swift、package-app.sh

第五阶段：收官加餐（第 14 篇）

篇	标题	你将学会	对应源码
14	加餐：投喂饥饿系统与省电优化	综合实战 + 省电三层防线，全书知识总复盘	AppDelegate.swift、PetController.swift

怎么读这套教程

- **按顺序读**：每篇只依赖前面的篇目，尤其 03（架构）是后面所有篇的地图。
- **对着源码读**：每篇开头标注了对应源码文件，先读教程讲解，再打开源码逐行对照——教程里贴的每一行代码都真实存在于项目中。
- **做每篇末尾的练习**：练习不是点缀，是「从看懂到会用」的唯一通道。每个练习都能在 30 分钟内完成。
- **随时跑起来**：`swift run`
  一秒钟看到效果，改一行代码立刻看到变化——这个项目最大的优点是反馈极快。

项目速览

```
FocusPet/
├── Package.swift # 构建清单（19 行）
├── package-app.sh # 打包脚本（26 行）
├── Sources/FocusPet/ # 全部源码（约 4000 行）
│   ├── main.swift # 程序入口
│   ├── AppDelegate.swift # 总指挥：装配一切
│   ├── PetController.swift # 宠物：窗口 + 行为 + 移动
│   ├── PetView.swift # 动画渲染 + 交互 + 气泡绘制
│   ├── FocusEngine.swift # 番茄钟状态机（141 行）
│   ├── StatusBarController.swift # 菜单栏图标（36 行）
│   ├── HotkeyManager.swift # 全局快捷键（70 行）
│   ├── SettingsModel.swift # 设置数据（123 行）
│   ├── SettingsView.swift # 设置界面（SwiftUI，180 行）
│   ├── StatsStore.swift # 统计存储（112 行）
│   ├── StatsView.swift # 统计界面（SwiftUI，95 行）
│   ├── Zodiac.swift # 星座计算
│   ├── Info.plist # 应用元数据
│   └── PetAssets/ # 144 帧 AI 生成序列帧素材
└── 教程/ # 你正在读的这套教程
```

几句心里话

这个项目从「能不能做个桌面宠物」的一句话，到上架 /Applications 的完整应用，全程没有一行代码是从模板里抄的——每个类、每个函数都是需求驱动、踩坑修正长出来的。这套教程记录的不仅是「怎么写」，更是「为什么这么写」：每一句注释里的教训（残影、漂移、点击无响应、透明洞），都来自真实发生过的故障。

编程最好的教材不是语法书，而是**一个你能完全看懂的、真实运转的程序**。现在你有它了。